LaTeX Indexing Editor

User Guide

LiDX

Table of Contents

List of Figures

Chapter 1

Introduction

The **LaTeX Indexing Editor** is a project-based editor for building and maintaining the index of a LaTeX document, built from `\index{...}` markers embedded in the source text. Instead of hand-typing and hand-placing the markup and hoping you got the syntax right, you insert entries from a dedicated panel and let the application generate the correct `\index` macro (hierarchy, page ranges, cross-references, sort keys, and page-number styling included) in the right place in the right file. The application provides a tree view and a spreadsheet-style table that show your entire index as it is being created, allowing you to edit the index content at any time.

Who is the guide for

Professional indexers working on a LaTeX-typeset publication, including indexers,

- with a non-LaTeX background using dedicated indexing tools like CINDEX, SKY Index, or Index Manager
- coming from marking up entries by hand in a word processor
- with experience in an indexing workflow where “LaTeX” and “`\index`” are new vocabulary rather than old habits

Chapter 2 is written specifically for that user; it assumes no prior LaTeX experience and explains the indexing model the rest of this guide builds on. If you already know LaTeX indexing, you can skip it and start at Chapter 3.

Organization of the guide

Chapter 2 is the conceptual primer — what a LaTeX index actually is and how it's built, independent of this application. Chapter 3 gets you from a blank slate to your first saved LaTeX `\index` entry.

Chapters 4 through 9 cover the application's interface in detail, one region or feature at a time: the Workspace Files Tree, the Index Tree, the Entry Table, custom LaTeX commands, the Tools menu, and a handful of additional features like Advanced Search and Name Inversion.

Chapter 11 covers Preferences, tying each setting back to the specific LaTeX package or engine it controls. Chapter 12 covers Troubleshooting, highlighting problems you may encounter, what the cause of the problem is and how you can fix each one.

The Appendix contains reference material; a syntax cheat sheet, the full keyboard shortcut table, an overview of the project databases, and a glossary.

How to use the guide

If you're new to LaTeX indexing, read it start to finish, each chapter builds on vocabulary and concepts the previous ones established, starting with Chapter 2. If you already know LaTeX indexing, or you're coming back later for a specific question, jump directly to the relevant chapter from the Table of Contents; the guide is written so that any chapter from 3 onward stands on its own.

Once you're familiar with the software and just need a quick answer, you don't need to come back to this document at all, the same material is also built into the application itself. **Help → Contents...** (or **F1**) opens a searchable topic browser covering every screen, panel, and dialog, organized to mirror the chapters here.

Chapter 2

Understanding LaTeX indexing

If you've built indexes with word-processor tools or dedicated indexing software before, LaTeX indexing will feel familiar in spirit but different in mechanics. Instead of composing the index in a separate tool that shows you a finished, page-numbered proof as you work, you embed markers directly in the document's source text. A separate program then collects, sorts, and formats those markers into the printed index at a later time.

This chapter explains that process from first principles, no prior LaTeX experience assumed,[1] so that when later chapters talk about the index tree, the entry table, a “range,” or a “cross-reference,” you already understand what the editor is actually doing underneath its interface, not just which button produces which result.

By the end of this chapter, you will be able to read a raw `\index{...}` macro in a `.tex` file and know exactly what it means, even without opening the application at all. If you already know LaTeX indexing, skip ahead to Chapter 3, Getting Started.

What is a LaTeX index, and how is it built?

A LaTeX index is built in three stages, spread across three different files that all share your document's base file name,

1. The .tex file(s): the document's source text, with \index markers scattered through it wherever a term should be indexed
2. The `.idx` file: a plain list of raw, unsorted entries, one per `\index` marker, written out automatically every time you typeset the document (Figure 1). Nothing is sorted or formatted yet; it's just a record of what was marked and on what page

```
\indexentry{welfare state|seealso{reciprocity beliefs; social solidarity}}{1}
\indexentry{income redistribution!public demands for|(}{1}
\indexentry{progressive taxation!self-interest}{1}
\indexentry{about}{2}
\indexentry{income redistribution!public demands for|)}{2}
\indexentry{benchmark model!analysis of}{3}
```

Figure 1. Example .idx file content.

3. The `.ind` file: the sorted, formatted, publication-ready index, produced by running a separate indexing program (usually `makeindex`, sometimes `xindy`) against the `.idx` file (Figure 2). This is what `\printindex` actually reads and typesets

[1] For a short introduction to LaTeX, see LaTeX for Authors. If you want a detailed reference to LaTeX, see LaTeX2e: An unofficial reference manual. Both these documents are maintained by the LaTeX team.

```
\item about, 2
\item actuarial fairness, 35, \fn{6}{39}
\item American Dream equilibrium, 56--57, 82
\item Aristotle, 38

\indexspace

\item belief change, \seealso{discursive context}{1}
  \subitem causal factors, 17, 20, 131--132, 158, 245
  \subitem economic shock, 126, 211, 212
  \subitem partisan dynamics, 130, 131, 160, 195--196, 202
```

Figure 2. Example .ind file content.

The indexing pipeline looks like this:

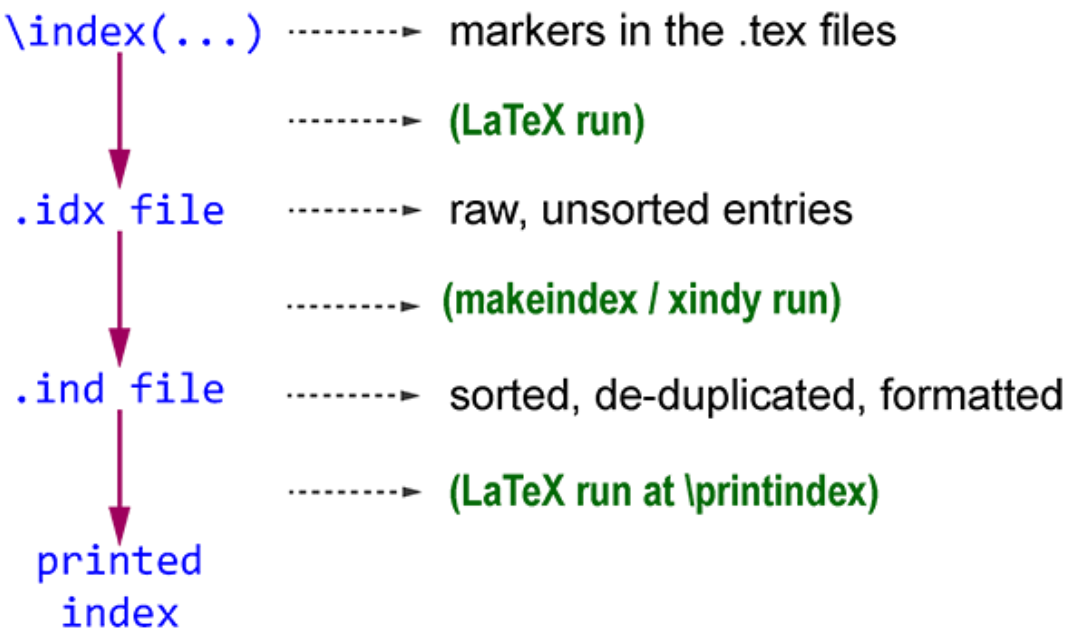

Figure 3. LaTeX indexing pipeline.

Two things follow directly from this pipeline.

First, LaTeX itself never sorts your index, that job belongs entirely to `makeindex` or `xindy`, a separate program which runs as its own step. Running LaTeX alone, no matter how many times, will never produce or update a printed index.

Second, because page numbers aren't finalized until LaTeX lays out the actual pages, and the index can't be typeset until `makeindex` has sorted it, producing a correct index normally takes several passes: LaTeX, then `makeindex`, then LaTeX again. You don't need to run these steps by hand (see “How the Editor Handles This for You” for what the editor manages for you) but understanding the process helps with understanding LaTeX preference selections.

Why bother with this rather than manually entering the index entries? For professional indexers, the appeal is precision and control. An index entry is a few characters of plain text living directly in the document source, so it can be reviewed, diffed, and version-controlled exactly like the prose around it, and its behavior is fully reproducible; the same source always produces the same index, on any machine, with no hidden state. The trade-off is that you're working with the underlying markup rather than a finished visual proof at every step. This is the gap the **LaTeX Indexing Editor** is built to close for you. It gives you tree and table views of your index while it manages the raw markers underneath.

The \index command

Every index entry starts as a call to the `\index` command, with the entry's heading as its argument:

```
...as defined by Adam Smith\index{Smith, Adam} in his 1776 treatise...
```

Two placement rules matter, both stemming from the same underlying cause, `\index` doesn't produce any visible output of its own, and the page number it acquires is simply whatever page LaTeX happens to be laying out at the exact point the macro appears in the source.

Placement: put `\index` immediately after the word or phrase it indexes, with no space in between, "`Smith, Adam\index{Smith, Adam}`", not "`Smith, Adam \index{Smith, Adam}`". An `\index` call is invisible in the typeset output of the manuscript, but has positional weight in the source. If a page happens to break right where a stray space sits before it, the index entry can be attributed to the wrong page.

No stray trailing space: if you write index entries on their own line for readability, end the previous line with a percent sign (`%`) so LaTeX doesn't read the line break itself as a space. Compare

```
text%
\index{term}
```

which is safe, against

```
text
\index{term}
```

with no `%` following `text`. This risks an unwanted space, and therefore a possible off-by-one page number[2].

You'll rarely need to worry about either rule directly, the editor always inserts a new `\index` macro at the exact source position of your text selection or cursor (see "Creating Your First Index Entry", and "Inserting Entries"), so correct placement and spacing are handled for you automatically.

Main entries, sub-entries, and sub-sub-entries

A single index entry can have up to three levels, a main heading, and up to two levels of sub-heading nested beneath it, separated inside the `\index` argument by an exclamation point (`!`):

[2] You should be aware of this rule primarily as it may relate to indexed legacy LaTeX documents you import into a project.

```
\index{birds}
\index{birds!songbirds}
\index{birds!songbirds!robin}
```

`makeindex` groups every entry sharing the same main heading together, and every sub-heading beneath its own parent, producing a nested printed index along these lines:

```
birds, 12, 45
  songbirds, 45
    robin, 45
```

This three-level structure (main, sub-entry, sub-sub-entry) is exactly what the editor's Index Entry window presents as three separate fields: **Main**, **Subhead 1**, and **Subhead 2** (see “Creating Your First Index Entry” and “Inserting Entries”).

Typing “birds” into Main, “songbirds” into Subhead 1, and “robin” into Subhead 2 and inserting the entry produces precisely the `\index{birds!songbirds!robin}` macro shown above — the fields and the `!` syntax are two views of the same thing, and the level cap of three is a hard limit of the underlying `\index` syntax itself, not an editor restriction (see "My Entry Disappeared" — the Three-Level Heading Limit,” for what happens if you try to exceed it).

Page Ranges

Most entries reference a single page, but a topic discussed continuously across several consecutive pages is better indexed as one range (42–47 for example) rather than as five separate single-page entries. `makeindex` supports this with a pair of range operators, `|(` to open a range and `|)` to close it, attached to the same heading:

```
Optimal foraging theory is introduced here.\index{foraging theory|(}
...several pages of continuous discussion...
This concludes our discussion of foraging theory.\index{foraging theory|)}
```

`makeindex` matches the opener and closer by heading text and collapses everything between them into a single range in the printed index (`foraging theory, 42–47`) rather than a list of page numbers. Get this wrong (an opener with no matching closer, two openers for the same heading before either closes, and so on) and `makeindex` will either warn you in its log or, in some cases, produce an index that fails to typeset at all.

The editor mirrors this exactly. Selecting a span of text before inserting an entry (rather than just placing your cursor) creates a linked opener/closer pair automatically, but displays it in the index tree as a single entry (the opener) so you interact with “one range,” not two separate macros (see “Range References”). Because ranges are two independent macros with no explicit link between them beyond matching text, they're also the most fragile part of an index, especially after a re-indexing pass; the Range consistency check tool exists specifically to find and repair the kinds of mismatches described above.[3]

Cross-References: See and See Also

As an indexer, you already know the difference between these two kinds of pointer: a *see* reference redirects the reader entirely (`Churchill, Winston, see Prime Ministers`) with no

[3] The application guards against these types of errors with ranged index entries it creates or when the user edits one. However, if you import a legacy LaTeX project that contains `\index` entries, or if a `.tex` file is edited outside the application, these errors may occur.

page numbers of its own, because everything relevant is filed under the target heading. A *see also* reference supplements rather than replaces (`Prime Ministers, see also Cabinet`) pointing the reader toward related material without denying that the current heading has its own real page references too.

In raw `\index` syntax, both are written as a special modifier attached to the heading with a vertical bar, in place of a page reference:

```
\index{Churchill, Winston|see{Prime Ministers}}
\index{Prime Ministers|seealso{Cabinet}}
```

Because a cross-reference carries no page number, it can technically be placed anywhere in the document without affecting accuracy, but it's easy for these to become scattered and inconsistent if entered by hand alongside ordinary content-based entries. This editor manages cross-references centrally, using a dedicated Cross-References tab and its own generated file, `cross_refs.tex`, kept entirely separate from your chapter files (see *The Cross-References Tab* and *Cross-References (See / See Also)*). Note also that `|seealso{...}` is not part of plain LaTeX indexing on its own, it needs a small supporting macro definition to typeset correctly, which is exactly what the editor's generated `cross_refs.tex` takes care of for you.

A note on existing work: a set of chapter files that was indexed before being brought into a project here will usually have its cross-references written inline, directly on an `\index` macro, in whichever file happened to contain them. The editor recognises these and offers to bring them into the managed system when the project opens, so you don't have to hunt them down by hand. That process is described in *Migrate legacy cross-references*.

Sort Keys vs. Displayed Text

Normally, the text of an index entry does double duty: it's both what gets printed and what `makeindex` alphabetizes by. Sometimes those two jobs need to pull apart, a name that must be inverted for filing purposes but not for display, a symbol or digit that should be alphabetized as if spelled out, or an entry whose displayed form contains LaTeX formatting commands that would otherwise confuse the sort. `makeindex` handles this with the `@` character, splitting an entry into `sortkey@displaytext`:

```
\index{Titanic@SS Titanic}
\index{L-function@\(L\)-function}
```

In the first example, the entry is alphabetized as “Titanic” but printed as “SS Titanic.” In the second, the entry sorts under “L” even though the printed form starts with a formatting command that would otherwise confuse the sort.[4] This `@` syntax can be applied independently at any of the three heading levels.

The editor exposes this split directly rather than asking you to type the @ yourself. The entry table carries paired Display and Sort columns for each heading level (see “Editing Entries”), and the Index Entry window offers a Sort as field beside each level as you create the entry — automatically once that level carries bold or italic, and on demand for any level. In both places, leaving the sort side empty files the entry under its display text, exactly as omitting the @ does in raw syntax. Nothing is generated on your behalf: what a level files under is either what you put in that field or the text itself.

---

[4] The \(and \) in the example are the LaTeX inline math delimiters that format the ‘L’ as a math symbol in italic. The printed output would look like “*L*-function.”

Styled Page Numbers (Bold / Italic)

It's a common indexing convention to mark one particular page reference for a heading differently from the rest (roman for the page where a topic gets its principal discussion, bold for a page containing an illustration or table, for example), so a reader scanning a long list of page numbers can spot the distinct one at a glance.

`makeindex` supports this with another use of the vertical bar, attaching the name of a formatting command (without a backslash or braces) after the heading, which wraps just that one page number (not the heading text) in the given command when the index is typeset:

```
Optimal foraging theory is defined here.\index{foraging theory}
...later, an illustration of foraging patterns.\index{foraging theory|textbf}
```

This produces “foraging theory, 12, **45**,” with 12 shown in roman and 45 in bold in the printed index, the styling command applies only to that specific page reference, never to every occurrence of the heading.

This is what the **Page Ref** control in the editor's Index Entry window sets (**Plain** / **Bold Page** / **Italic Page**, defaulting to Plain) and what the **Page** column shows in the entry table; the editor writes |textbf or |textit for you, so you never need to type either by hand. Don't confuse this with the **B** / **I** Text Style buttons in the same window, which format part of the heading text itself, not the page number.

Escaping Special Characters

You've now met every character `makeindex` reserves for its own syntax, each doing double duty as an ordinary character you might also want to type literally into a heading:

Table 1. Special characters.

Special Character	Description
@	separates a sort key from display text
!	separates heading levels
|	introduces a cross-reference or a styling encapsulator
"	`makeindex` escape character
****	escape character for LaTeX itself

Because each of these already has a special meaning, none of them can simply be typed as-is when you want the literal character rather than its special behavior. `\index{gnat!}` with a trailing `!` intended as punctuation would instead be read as an attempt to open a heading level with an empty name.

`makeindex` resolves this with a single escape convention, prefix any of the first four with a quotation mark (`"`) to force it to be treated as an ordinary character instead of triggering its special meaning.

\index{one"@two}	\indexentry{one@two}	(literal @)
\index{three"|four}	\indexentry{three|four}	(literal |)
\index{five"!six}	\indexentry{five!six}	(literal !)

A literal quotation mark is the one special case: since `"` is itself the escape character, producing one takes two of them in a row (`""`), not a single one preceded by an escape marker:

```
\index{f""ur}  ->  \indexentry{f"ur}
```

This matters most for a kind of entry professional indexers write constantly, with no dedicated LaTeX syntax of its own; an article, chapter, or song title conventionally set off with quotation marks in the index, as distinct from a book title in italics. An entry meant to display as

```
“The Old Man and the Sea”
```

— sorted under “Old,” not “The” — is written with each of the two literal quotation marks doubled, combined with the ordinary sortkey@displaytext split:

```
\index{Old Man and the Sea@""The Old Man and the Sea""}
```

None of this escaping is automatic in the editor. Text typed into the Main / Subhead 1 / Subhead 2 fields is passed through to the underlying `\index` argument largely as-is, the one exception is the automatic sortkey@displaytext generation triggered by the B/I Text Style buttons. A literal `@`, `!`, `|`, or `"` typed directly into a field needs the escape convention above applied by hand before you insert the entry. This is also the actual fix for the situation described in “"My Entry Disappeared" — the Three-Level Heading Limit”, an unescaped `!` in a field is read as an ordinary (unintended) level separator. Type `"!` instead if a heading genuinely needs a literal exclamation point.

It does, however, tell you when it matters. Every field in the **Index Entry** window is checked as you type, and a small icon appears at the right-hand end of any field whose text holds a character LaTeX or makeindex will read as grammar rather than as text. Hover it for one line per problem, in plain terms; click it to apply the escapes to the whole field at once, and `Ctrl+Z` to change your mind. The same icons appear beside cells in the entry table, where they report but do not repair. What has not changed is that nothing is escaped behind your back — the icon offers, and the entry inserts either way.

One further wrinkle, useful rather than hazardous: a backslash typed into a field is preserved by makeindex, but still means something to LaTeX afterward. That's exactly how a LaTeX accent command survives being written into an index entry unescaped — `\index{f\"ur}` passes straight through to `\indexentry{f\"ur}`, later typeset correctly by LaTeX as “für.” That one works because a backslash placed before makeindex's own escape character is defined to make it literal. **Do not generalize from it: a backslash protects nothing else.** makeindex has no use for a backslash at all — it copies one verbatim into the generated index for LaTeX to interpret, which is precisely why `\%` and `\&` work — so `\!`, `\@` and `\|` are not escapes at all, and `\index{A\|B}` really does come out with a page style of B. Only the quotation mark suppresses `!`, `@` and `|`. Ordinary LaTeX-reserved characters (`& $ # % _ { } ^ ~`) are a separate matter entirely, needing their normal LaTeX escaping regardless of indexing — and LaTeX still requires balanced braces even inside an `\index` argument.

How the Editor Handles This for You

Everything in this chapter is genuinely happening underneath the editor at all times, every entry you create really is a `\index{...}` macro, in the exact syntax shown above, sitting in your `.tex` source. Nothing about the underlying LaTeX indexing model is hidden from you or replaced with a proprietary format; if you opened your project's files in a plain text editor, you'd find ordinary, hand-writable LaTeX markup.

What the application removes is the tedium and error-proneness of producing that markup correctly by hand; exact macro placement and percent-sign spacing, the `!` and up-to-three-level structure, matched opener/closer pairs for ranges with a dedicated consistency checker for when they drift, centrally-managed cross-references instead of scattered inline pointers, the `sortkey@displaytext` split offered as its own field wherever a heading is edited, and page-number styling instead of memorized `|textbf`/`|textit` syntax. Escaping is the one piece of this chapter's raw syntax that still has to reach the field as literal characters — the application will not insert them behind your back — but it no longer leaves you to notice on your own: a literal `@`, `!`, `|` or `"` in a heading is flagged where you typed it, with the correction a click away.

Chapter 3

Getting Started

This chapter takes you from nothing installed to your first saved, working index entry. If you haven't read Chapter 2 yet and you're new to LaTeX indexing, it's worth doing that first, this chapter assumes you already know what a `\index` macro is and what it's for; it's focused entirely on the application itself.

Installing and Launching the Application

Installing the executable

Download the Windows installer from the project's GitHub Releases page (`https://github.com/DWHowes/LaTeX_Indexing_Editor/releases`), pick the newest release and run its `.exe`. No Python, no command line, and nothing else needs to be installed first; it installs just for your own user account, so no administrator rights are needed either.

Windows may show a “Windows protected your PC” SmartScreen warning the first time, this is expected for an early alpha build that isn't yet code-signed, not a sign anything is wrong. Click “More info,” then “Run anyway” to proceed with the install.

Once installed, find **LaTeX Indexing Editor** in your Start Menu (or on your Desktop, if you chose that option during setup) and open it. There's no project open yet at this point, see Opening and Creating a Project for how to do this.

Running from source

For development work, or on a platform other than Windows (the packaged installer is Windows-only), the application can be run directly from its Python source. This needs Python 3.13 or later and the ability to install a handful of packages with `pip`. An example script to clone the repo and install the necessary packages is,

```
git clone https://github.com/DWHowes/LaTeX_Indexing_Editor.git
cd LaTeX_Indexing_Editor
python -m venv .venv
# Windows (PowerShell):
.venv\Scripts\Activate.ps1
# macOS/Linux:
source .venv/bin/activate

pip install -r requirements.txt
```

Once the virtual environment is active and dependencies are installed, `python main.py` starts the application the same way the installer's shortcut does. An example Windows batch file to run the application is,

```
@echo off
cd /d <installation path on your computer>
call .venv\Scripts\activate
```

```
python main.py
pause
```

Application Overview (Interface Tour)

The main window is split into a few fixed regions. This section just orients you to what's where; every region gets its own full chapter later.

Menu bar

The menu bar (Figure 4) contains the following;

- **File**: open/open recent/save/close a project, exit
- **Edit:** Find, Advanced Search, inserting LaTeX settings and custom commands into the base file, Preferences
- **View:** switching which sidebar panel is showing, toggling the Index Entry window
- **Tools:** head notes, custom LaTeX commands, RTF export, resyncing, pruned file
- **Help**: display the help file

Figure 4. Application menu bar.

Toolbar

Located just below the menu bar (Figure 5) it contains;

- a dark/light mode toggle
- three buttons that choose which panel the left sidebar is currently showing; **Workspace Files**, **Index References** (the Index Tree), and **Edit Entries** (the Entry Table). Only one of the three is visible at a time
- font family/size controls that apply cross-project (globally) to the text in editor tabs[5]
- application close button

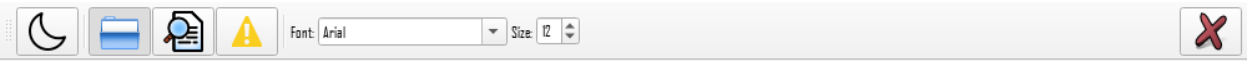

Figure 5. Application tool bar.

Views sidebar

This sidebar (Figure 6) displays the view selected from the toolbar, or the tab clicked by the user. The sidebar is composed of;

The **Workspace Files** tree (Chapter 4): allows the user to browse and manage every `.tex` file in the project

The **Index Tree** (Chapter 6): shows the whole index as a hierarchy and allows the per-node editing of index entries

[5] This is display formatting in the application editor tabs only. Text styling of the PDF output is controlled by LaTeX commands.

Edit Entries (Chapter 7): a spreadsheet-style table view, with its own Cross-References sub-tab. Provides for the per-reference editing of index entries

Figure 6. Views sidebar.

Editor tabs

Displays the content of a `.tex` file in a closable, reorderable tab with limited editing capability (Figure 7). Index entries are inserted into the active tab.

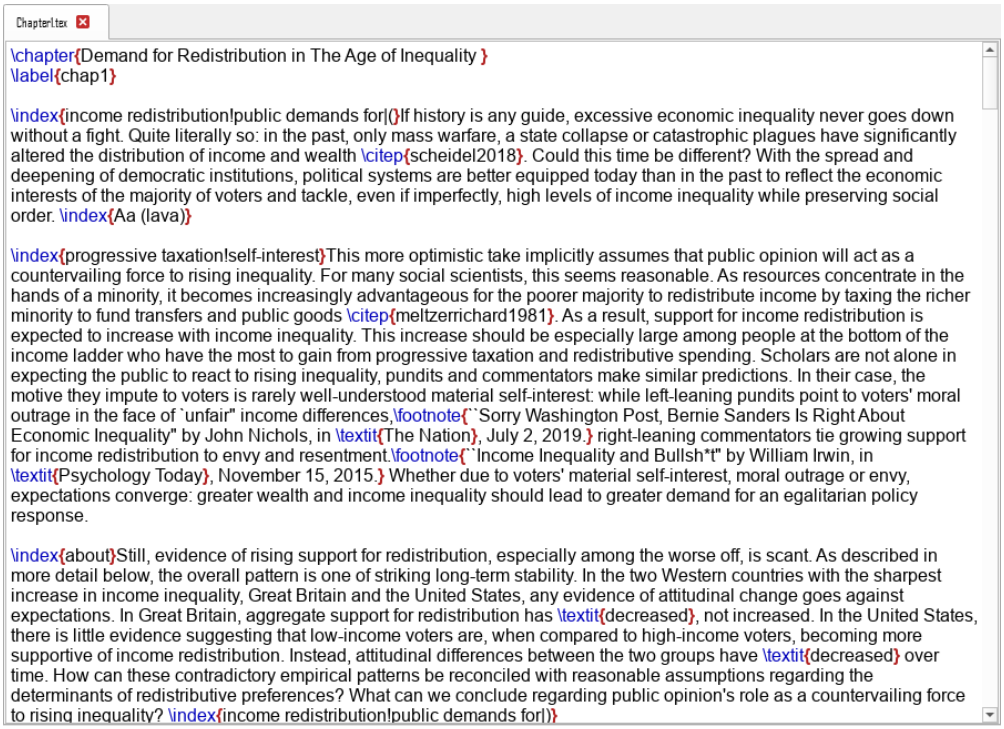

Figure 7. Editor tab.

Index Entry window: a docked panel along the bottom of the editor tabs (Figure 8), hidden until you toggle it (**View** → **Toggle Index Entry Window**, `Ctrl+\`). This is where you actually build and insert a new `\index` entry.

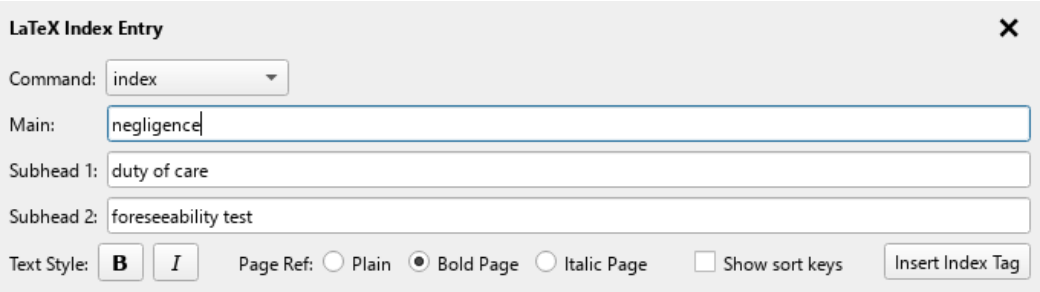

Figure 8. Index entry window.

Status bar: along the very bottom of the window: brief feedback messages (a save confirming, a tool reporting no problems found, and so on) and warnings about actions that aren't currently available (for instance, why a Tools menu item is disabled if no base file has been chosen yet).

Opening and Creating a Project

A **project** is a folder containing LaTeX source files, plus one small database file the editor uses to keep track of every `\index` entry it finds in them. One command (Figure 10) handles both opening an existing project and creating a new one: **File** → **Open Project...** (`Ctrl+O`).[6]

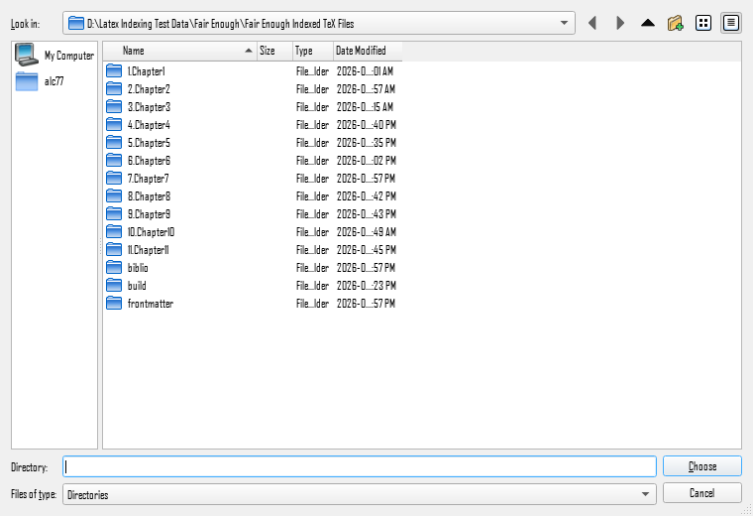

Figure 9. Open project dialog box.

[6] The first time this dialog is called, it defaults to the application installation directory. However, once you have created a project, the dialog defaults to the last selected directory. This makes re-opening a project simple.

Opening an existing project

Choose **File** → **Open Project...**, select the folder that contains the `.tex` files, and if the editor finds a project database already set up there, it opens immediately — you won't be asked anything further. If another project is currently open, you're prompted to save or discard any unsaved changes first.

Reopening a recent project

File → **Open Recent** lists the projects you have opened before, most recent first, so returning to one does not mean navigating to its folder again (Figure 10). The first nine can also be chosen by number once the submenu is open. Each entry shows the project's name; hovering over one reveals the full path, which is what distinguishes two projects whose folders happen to share a name. Choosing an entry behaves exactly like opening that project the long way, including the prompt about unsaved changes in whatever is currently open.

A project appears in this list only after it has opened successfully, so a cancelled or failed open leaves no trace, and reopening a project moves it back to the top rather than adding a second copy. If you choose a project whose folder has since been moved or deleted, the editor says so and offers to remove it from the list. How many projects are listed, whether the submenu appears at all, and a button to clear the list are all in **Preferences** → **General**.

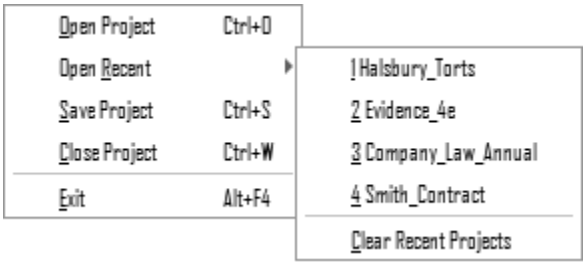

Figure 10. The File menu's Open Recent submenu.

Creating a new project

Select a folder that doesn't already have a project database, and the editor treats it as new. You'll be prompted to enter a project name, it defaults to the folder name but doesn't need to match it.[7] The editor then creates its database file in that folder and scans every `.tex` file already inside it for existing `\index` entries.

This step is worth examining in more detail. Creating a “new” project over a folder that already contains legacy indexed content (a manuscript indexed by another tool or by hand) is completely normal and expected. Nothing in the folder is modified by this scan; the editor only reads what's already there. In this case you end up with a fully populated Index Tree the first time the project opens, not an empty one.[8]

---

[7] When entering a project name, only letters, numbers, spaces, underscores, and hyphens are kept. Spaces are translated into underscores.

[8] As the application scans all `.tex` files when creating a new project, no user action is needed to import a legacy LaTeX index, it happens automatically if `\index` tags are present in the files.

The database file itself (`<ProjectName>_index_manifest.db`, created alongside the `.tex` files) isn't something you need to open or edit yourself. It's what makes reopening a large project fast, and what tools like **Index Statistics** and **Range Consistency Check** query directly. Because the editor trusts this database rather than re-scanning the folder on every open, a file added, removed, moved, or edited outside the application won't be picked up automatically, **Resyncing Workspace Files from Disk** is how you catch it up, rather than deleting and recreating the database by hand.

The Base File

The **base file** (also called the **root file**) is the one `.tex` file in your project that actually gets compiled. This file contains `\documentclass{...}` and `\begin{document}...\end{document}`, tags and is the container into which every chapter or section file is pulled via `\input/\include`. Several features write directly into this specific file or compile it; **Insert LaTeX Index Settings**, **Custom LaTeX Commands**, **Head Notes**, the **Cross-References** tab, and **RTF Export**, so the editor needs to know which one it is before any of those can work.

- **Automatic detection:** each time a project is opened without a base file already recorded, the editor scans every active file looking for one that contains both `\documentclass{...}` and `\begin{document}`. If exactly one file matches, it's set automatically — no action needed. If zero or more than one file matches (several standalone chapter drafts, each with its own preamble, for example), detection is ambiguous and nothing is set; you'll choose manually.
- **Setting it manually:** right-click any `.tex` file in the Workspace Files tree and choose **Set "‹filename›" as root file**. This works whether or not a base file was already chosen — picking a different one simply replaces the previous choice — so it's also how you correct a wrong automatic guess.
- **Visual identification:** the base file is shown in **bold**, highlighted in the Workspace Files tree. If nothing is highlighted, no base file has been chosen yet, and any feature that needs one will say so in the status bar rather than silently doing nothing.

Like most settings in this application, the base file is stored per project, so it's remembered the next time you open it, and choosing one has no effect on any other project.

Creating Your First Index Entry

With a project open and a base file chosen, here's the full path from a blank slate to your first entry. This example builds directly on the `\index` syntax introduced in Chapter 2, so you can see exactly what ends up in the `.tex` file (Figure 11).

1. **Open a file:** from the **Workspace Files** tree, double-click a file to load it into an editor tab.
2. **Select the text you want to index:** A cursor placement with no text selected produces a point reference; selecting a span of text produces a page-range reference. For a first entry, just place your cursor after a word, no selection needed. Remember to place the cursor immediately after the word with no intervening space.
3. **Open the Index Entry window: View → Toggle Index Entry Window** or `Ctrl+\`, if it isn't already showing. Focus jumps straight to the **Main** field.

4. **Type a heading into Main:** this becomes a top-level entry.[9]
5. **Insert the index entry:** click **Insert Index Tag**, or press `Ctrl+K`.

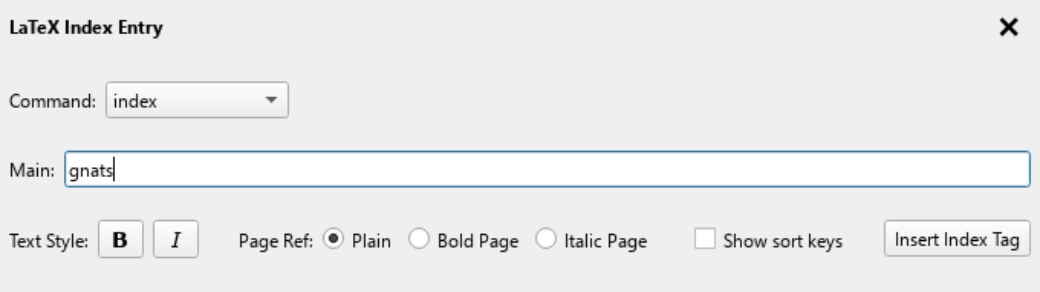

Figure 11. Example index entry window.

That's it, the entry now exists. If your cursor was, say, right after the word “gnats” and you typed `gnats` into Main, the editor has just written this into the `.tex` file at exactly that position,

```
gnats\index{gnats}
```

You will see the placement of the new index entry in the active editor tab. The entry also appears immediately in the **Index Tree** (Chapter 6) and the **Entry Table** (Chapter 7).

From here, try filling in **Subhead 1** as well before inserting (to produce a two-level `main!sub`), or selecting a span of text first instead of just placing your cursor (to produce a range), the panel resets after every insert, ready immediately for the next one. Chapter 5 covers every control in the panel, including the **Sort as** fields that decide where a heading files.

Saving and Closing

What counts as “unsaved” here is broader than it looks. Everything the project database records about your index — new entries, edited headings and sort keys, page-style changes, deleted references — is held in memory as you work and written to the database in a single batch when you save. Nothing about your index reaches the database before that.

The `.tex` side depends on whether the file is open. A file open in an editor tab takes the change into that tab's buffer, and the buffer reaches disk when you save — which covers every entry you insert, since insertion always goes into the active tab. A file with no open tab is rewritten on disk as you make the change, which is how a heading rename sweeps through files you never opened; its database rows still wait for the save.

Three things never wait for a save: cross-references created in the **Cross-References** tab, along with the regenerated `cross_refs.tex`; head notes; and all project configuration — the base file, pruned files, adopted custom commands, and everything in **Preferences**.

Save

File → **Save Project** (`Ctrl+S`) does four things, in order. It writes every open editor tab's buffer out to its file. It writes all pending index changes to the project database in a single transaction, so either the whole batch lands or none of it does. It re-stamps the checksum it keeps for each

[9] If you accidentally hit ENTER and display the entry box for the first sub-heading, no harm is done. You can either just leave it empty or hit the BACKSPACE key to close the box.

file, so your own work isn't reported back to you as an outside change the next time the project opens. And it clears the session's backup snapshots, which are no longer needed as a fallback once everything is safely written.

If the database write fails, the status bar says so and nothing at all is written — your changes are still there, still pending, and the next save tries again. Because index changes accumulate in memory between saves, it is worth saving regularly through a long editing run.

The editor also saves for you on a timer — every 5 minutes by default, adjustable or switchable off under **Preferences** → **General** (10.1). An automatic save is exactly the same operation as `Ctrl+S`, which means it moves the Discard point too. It stays out of the way otherwise: a tick with nothing to write does nothing, a tick arriving mid-dialog or mid-edit waits for the next one, and it never interrupts with a dialog of its own. It reduces what a crash can cost without removing the reason to save at a natural stopping point.

Closing a project

File → **Close Project** (`Ctrl+W`) walks through every open editor tab one at a time. For each tab with unsaved changes, you're asked to **Save**, **Discard**, or **Cancel** (Figure 12). **Discard** reverts that one tab's file to how it stood at the start of the session, **Cancel** stops the close entirely (any tabs already resolved keep whatever you chose).

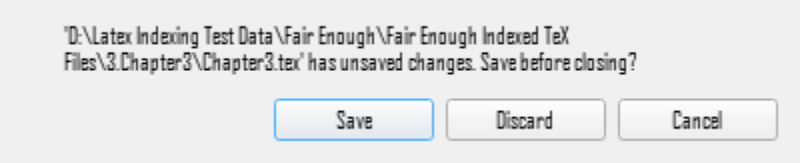

Figure 12. Save dialog displayed when closing a project with modified files.

Once every tab is resolved, one further prompt appears if any index changes are still unwritten, **Unsaved Index Changes**. This catches what no tab prompt could have covered — an edit to a file you never opened leaves no modified tab to ask about. **Save** writes them, **Discard** abandons them and puts the affected files back to their session-start content, so the source and the database can't be left disagreeing, and **Cancel** leaves the project open. Opening a different project closes the current one the same way, prompts included.

Once everything is settled, the project closes: the **Index Tree** and **Entry Table** clear, and project-specific menu items become unavailable until another project is opened.

Exiting the application

Closing the window, or **File** → **Exit** (`Alt+F4`) works a little differently (Figure 13). Instead of asking about each tab individually, you get a single prompt covering the whole workspace.[10] **Save** and **Discard** there apply to everything at once.

[10] This prompt is triggered if anything is unsaved anywhere at all, including an index entry inserted this session that hasn't been part of an explicit save.

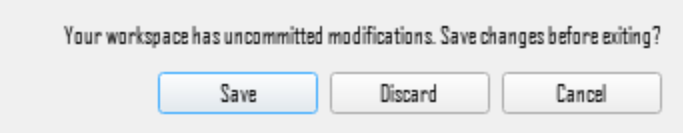

Figure 13. Save dialog displayed when closing the application with modified files.

Under the hood

The first time the editor writes to a file during a session, it keeps a backup of that file's original, pristine content. **Discard**, whether for one tab or the whole workspace, restores from that backup rather than trying to undo individual edits, so it always gets you back to exactly where a file stood when the session started touching it, no matter how many changes happened in between. These backups are cleared automatically once you save.

A save also moves the line **Discard** rolls back to. Because saving clears the backups, **Discard** always returns a file to its state at the last save — automatic or manual — not to the moment the project was opened.

If the application closes unexpectedly, index changes that hadn't been saved are gone — they only existed in memory — and so are the entries you inserted into an open tab. Changes to files with no open tab are the asymmetric case: their `\index` macros are already on disk, but the matching database rows were never written. The editor notices this the next time the project opens, comparing each file against the checksum it recorded and raising **Files Changed**, which offers to resync the index data from those files.

Opening a Project Made by an Earlier Version

A project's database carries a record of the shape it was written in, and opening a project made by an earlier version of the application brings that database up to date before anything else happens. You are not asked, because there is nothing to decide: the alternative to migrating is not opening.

Each step is applied in order and stamped only after it has actually run, and the whole sequence happens inside one transaction. If any step fails, every change and every stamp is rolled back together, so a database is never left claiming to be a version it did not reach. A failed migration leaves your project exactly as it was.

There are two independent sequences, because there are two owners. The tables holding your file list, your checksums and your project's custom commands belong to this application; the tables holding the index itself, the cross-references and the project settings belong to the shared indexing core the application is built on. Each migrates on its own list under its own version key, so one can gain a table without the other appearing to have changed.

Your .tex files are not touched by any of this. They are the source of truth and the database is a cache over them; in the worst case a database can be deleted and rebuilt by reopening the project, and the cost is the time of a rescan rather than any of your work.

Chapter 4

The Workspace Files Tree

The Workspace Files tree (Figure 14) is the left sidebar's default panel (**View** → **Focus File Pane**, `Ctrl+B`), the first thing you see when a project opens. It lists every `.tex` file the editor currently tracks as part of the **project**, arranged the same way the files sit on disk, and is where three project-level decisions happen,

- opening a file into an editor tab
- choosing which file is the base file
- excluding a file from the project's indexing scope (pruning) without deleting it

This chapter covers; browsing and opening files, setting the base file manually, pruning files that don't need indexing, and the manual resync action that reconciles the tree with what's actually on disk.

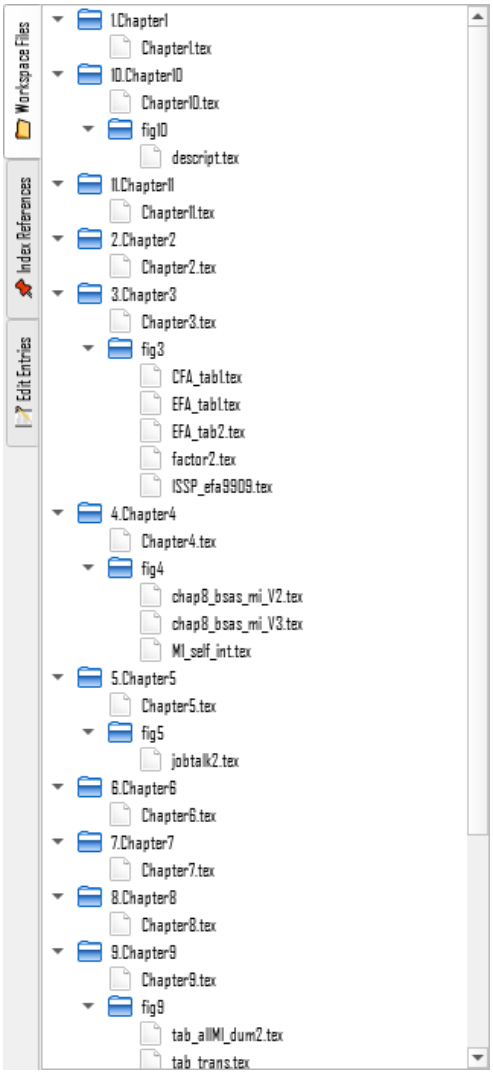

Figure 14. Workspace Files tree view.

Browsing Project Files

The tree mirrors the project folder's real structure (folders and files) sorted alphabetically within each folder, the same as any normal file browser. A folder or file icon marks which is which, and clicking the arrow beside a folder expands or collapses it like any standard tree view.

Double-clicking a file loads it into an editor tab, or brings its tab to the foreground if it's already open. Double-clicking a folder does nothing beyond the ordinary expand/collapse — there's no rename, move, or delete from this tree; it's a navigation and project-scope viewer, not a general file manager.

The project's **base file**, once chosen, is shown in bold with a highlight colour so it stands out at a glance.

Setting the Base File

Beyond the automatic detection described earlier, you can set, or change, the base file directly from this tree at any time: right-click any `.tex` file and choose **Set “‹filename›” as root file**. This works whether or not a base file was already chosen, picking a different one simply replaces the previous choice, which is also how you correct a wrong automatic guess or switch base files if your project's structure changes.

The change takes effect immediately; there's nothing to save. The status bar confirms with `Root file set successfully.`, and the tree's bold highlighting moves to the newly chosen file right away.

Pruning Files

Not every `.tex` file in a project needs indexing; a bibliography file, a class or style file living alongside your chapters, a file containing no indexable text. Right-click any file other than the current base file and choose **Prune “‹filename›” (Contains No Index Text)** to remove it from the project's active scope. It disappears from the tree, and the editor no longer scans it for `\index` entries or includes it in **Advanced Search**.

The base file itself can't be pruned this way. Prune isn't offered for it in the right-click menu, since a project always needs one.

Pruning never touches the file on disk. Nothing is deleted, moved, or edited, only a record inside the project's database is marked inactive, which is why it's fully reversible.

Restoring Pruned Files

If you’ve made a mistake in pruning or decide to reinclude some files, **Tools → Manage Pruned Files...** (Figure 15) opens a checklist of currently pruned file, with each file pre-checked. Uncheck anything you don't want back, or use Select All / Select None, then Restore Selected. Restored files reappear in the tree immediately, back in active scope exactly as they were before being pruned.

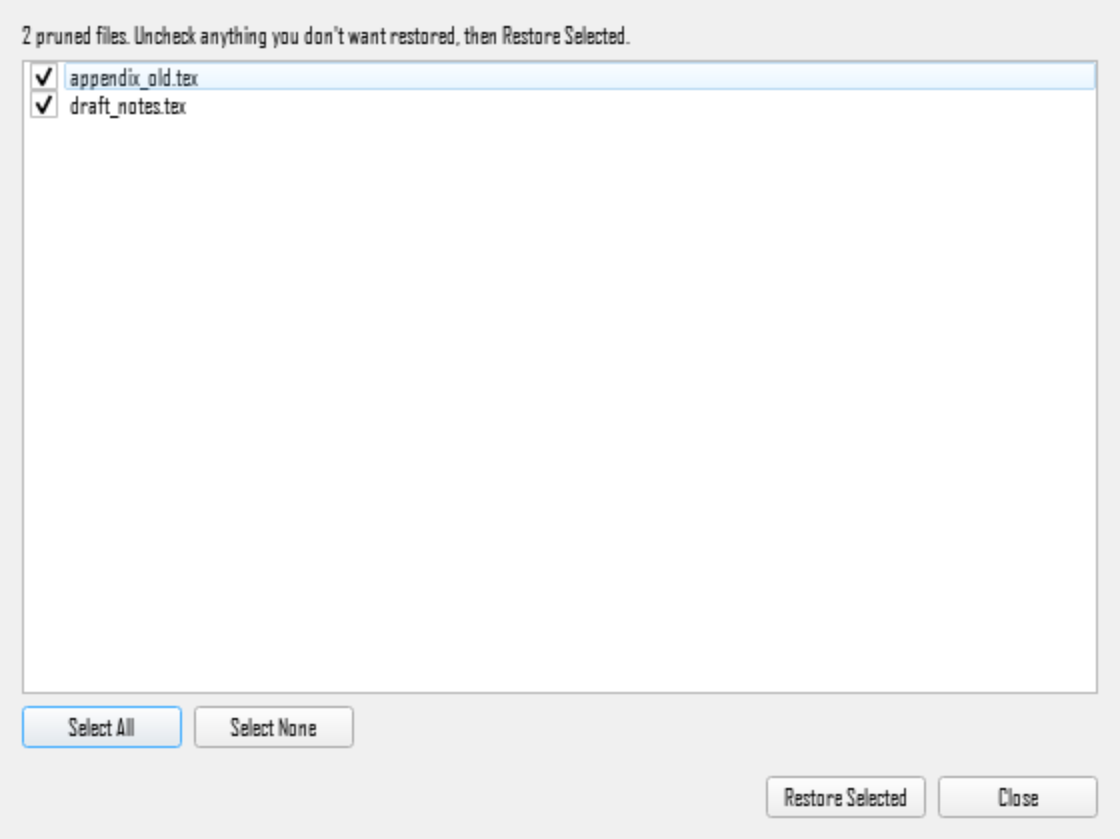

Figure 15. Restore pruned files dialog.

Resyncing Workspace Files from Disk

Once a project has been opened at least once, the **Workspace Files** tree trusts its own database rather than re-scanning your project folder on every subsequent open, this is what makes reopening a large project fast. The trade-off is that a file added, removed, or moved outside the application; in Explorer, another editor, or a sync tool, isn't picked up automatically. The tree keeps showing whatever it last knew, until you tell it otherwise.

Tools → Resync Workspace Files from Disk is the tool that brings the database back in concert with the physical disk file tree. It re-walks the project folder and reconciles the tree against what's actually there, newly discovered files are added, and tracked files no longer present on disk are dropped.

One side effect worth knowing before you use it: resyncing also un-prunes any previously pruned file that's still sitting on disk. From the resync's point of view, “still exists” and “should be active” are the same thing, so a file you deliberately pruned will reappear unless you've also deleted or moved it away.

This is a workspace-files-only operation — it doesn't touch index content at all. If your goal is instead to catch up on `\index` entries edited into an already-tracked file from outside the application, that's **Resync Index Data from Disk**, a separate, heavier operation with its own trade-offs.

Chapter 5

The Index Entry Window

Every entry in your index begins in one panel. The **Index Entry window** takes a heading, up to three levels deep, and writes it into your source as a real `\index` macro at the cursor — or wrapped around whatever you had selected. The panes described in the chapters either side of this one show you entries that already exist; this is where they come from.

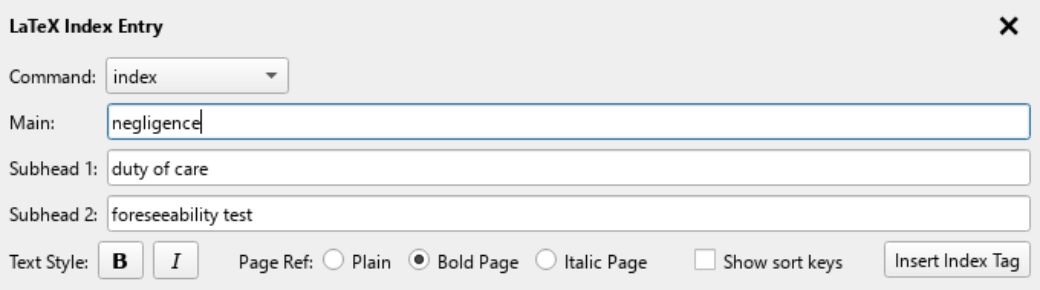

Figure 16. The Index Entry window.

Nothing in the panel is written until you press **Insert Index Tag**, and what it writes is exactly what the fields say — the editor does not add sort keys, formatting or page styles of its own accord.

Opening and Closing the Panel

View → **Toggle Index Entry Window** (`Ctrl+\`) shows and hides it. The action stays greyed out until a project is open, since there is nothing to index without one. The panel docks along the bottom of the editor tabs and carries its own slim title bar with a **×** button; pressing **Esc** while it has focus closes it just as the button does.

Whenever the panel appears, the **Main** field takes focus, so a session of steady indexing needs no mouse: show the panel, type, insert, repeat.

Choosing the Command

The **Command** dropdown at the top left decides which macro wraps the entry. It defaults to **index** — plain `\index`, which is what most projects want and what the rest of this chapter assumes.

A project that indexes through a macro of its own can adopt that command, after which it appears in this list. Only commands the current project has adopted are offered, and only those whose definition is a `\newcommand`, `\renewcommand` or `\providecommand` that actually calls `\index` — a command can be adopted and still not appear here if it does not meet both conditions. Custom commands are covered in their own chapter.

The Three Heading Levels

Main is always visible. Type the top-level heading and press **Enter**: a **Subhead 1** field appears beneath it, already focused. Press **Enter** again with text in that field to reveal **Subhead 2**. The levels appear as you need them rather than all at once, so an ordinary single-level entry is one field and one keystroke.

Changed your mind about a sub-level? Press **Backspace** in an *empty* Subhead field. It collapses out of view and hands focus back to the field above, taking its sort field with it.

Three levels is the limit, and it is `makeindex`'s limit rather than the editor's — a fourth `!` in a heading does not produce a fourth level, it produces a heading with a stray character in it. The concepts chapter explains why; the troubleshooting chapter covers what it looks like when an entry goes missing because of it.

All three fields suggest headings already used at that same level elsewhere in the project. Suggestions appear once you have typed two characters, and **Tab** accepts the highlighted one. This is worth using: it is what stops “fairness beliefs” and “Fairness beliefs” becoming two headings that have to be merged later. A suggestion that carries a sort key brings the key with it, filling both fields.

Styling Text Within a Heading

The **B** and **I** buttons format *part of a heading*. Select some text in whichever field you are working in and click one: the selection is wrapped in `\textbf{...}` or `\textit{...}`, which is what will print in the finished index. A book or case name inside an otherwise plain heading is the usual reason.

They never touch a sort key — they grey out while a **Sort as** field has focus. A sort key is read by the indexing engine and never printed, so there is nothing in it to style. They have nothing to do with **Page Ref** either, which styles the page number rather than the heading.

Your selection is widened first if it has to be. A text field will let you select any two points, including the middle of a command you have already applied, and wrapping that literally produces markup that builds and then prints wrongly — a doubled backslash is a line break, and a command name stripped of its backslash prints as an ordinary word. So before anything is wrapped the selection grows until a command can safely take it as an argument: a command is never cut in half, it keeps its argument group, and braces balance. With `RMS \textit{Titanic}` in the field, selecting just the backslash and selecting from just after it into the middle of the word both give you `RMS \textbf{\textit{Titanic}}`, and the status bar mentions that it widened. Selecting only the words inside a group still formats only those words, and a field whose braces do not balance is declined outright with a note saying why.

Sort as: Filing a Heading Where It Belongs

The text that prints and the text a heading files under are not always the same, and formatting is where they part company most often. A **Sort as** field appears beside a level as soon as that level carries bold or italic, because that is the case where the printed form cannot be filed on at all: `makeindex` would sort `\textit{The Quality of Mercy}` on the backslash, landing it among the symbols before the letter A.

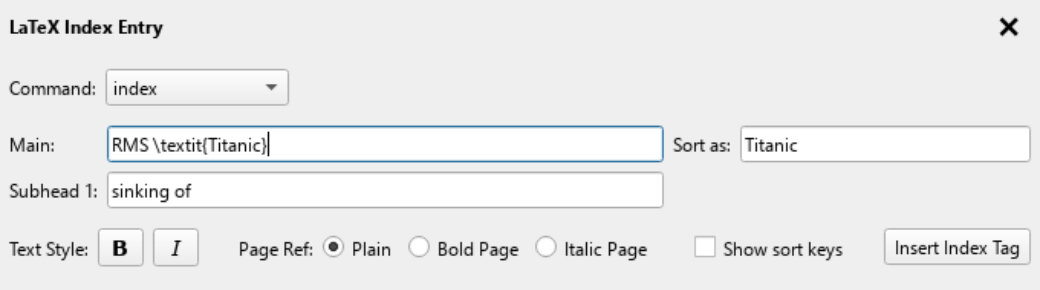

Figure 17. A sort key offered on a level that carries italics.

The field starts out holding the heading with its formatting read out, and keeps up with the display text while you leave it alone. **Type in it once and it is yours** — nothing rewrites it afterwards. That first edit is usually the whole point, because reading the formatting out is not the same as knowing where a title files:

- `\textit{The Quality of Mercy}` reads as *The Quality of Mercy* and files under **Q**. An indexer drops the article; the editor cannot know to.
- `RMS \textit{Titanic}` reads as *RMS Titanic* and files under **T**. The vessel prefix is not what a reader looks under.

Sort keys are not only about formatting, and the **Show sort keys** switch puts the field on every level whether it carries formatting or not — *St. John* filed under *Saint John*, *1984* filed under *Nineteen Eighty-Four*, a symbol filed as if spelled out. The switch is remembered between sessions, so an indexer who works this way sets it once.

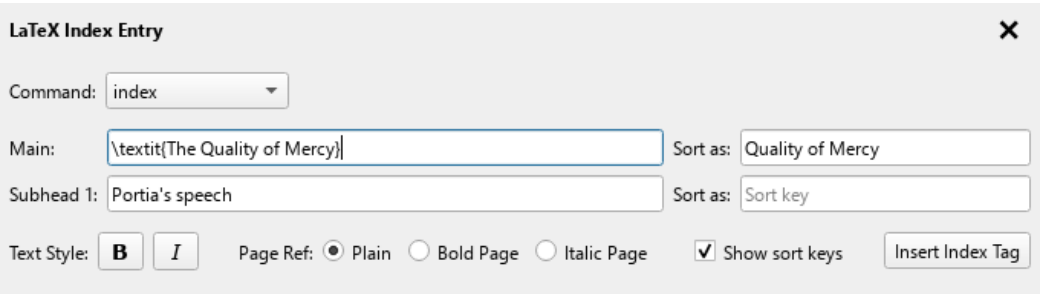

Figure 18. Show sort keys reveals the field on every level.

Leaving the field empty is a decision, not an omission. An empty sort field files the level under its display text exactly as written, formatting included, which is occasionally what you want and never what you want by accident — so when a formatted heading goes in with no key, the status bar says so once and inserts it anyway. Nothing is generated on your behalf.

This is the same `sortkey@displaytext` split described in the concepts chapter, and the same one the entry table exposes as its paired Display and Sort columns. Both write identical tags, and the entry table is where to correct the sort key of an entry that already exists.

Typing an `@` into a heading field splits it here too, the moment you leave the field. It is makeindex's own sort-key separator, so `Titanic@RMS \textit{Titanic}` moves apart into the two fields it means — which is also how an autocomplete suggestion carrying a key gets unpacked. When that is not what you meant, and `user@host` is the obvious case, the status bar names the level it happened on and a small undo button appears at the right-hand end of the

field. One click puts the text back exactly as typed and empties the sort field again; that text will not be split a second time unless you edit it into something else.

All six fields are checked as you type — the three heading levels and their three **Sort as** fields — and carry a warning icon whenever their text holds a character that will not mean what you meant. A triangle is the serious one: the document will not build, or the entry is silently lost or truncated. The case worth knowing is a bare `%`, which starts a LaTeX comment — `\index{Profit % margin}` compiles with no warning anywhere, and the printed index shows just Profit, with no page number. An information icon means it will build but will say something else: `Bang! Goes` becomes two heading levels, and `a|b` reads b as a page style. Click the icon to correct the whole field at once, and `Ctrl+Z` to change your mind. A finding with no mechanical fix — an unmatched brace, a trailing backslash — greys the icon and explains itself in the tooltip instead, because those need you to say what you meant. Nothing is blocked either way: this is advice, and the entry inserts regardless.

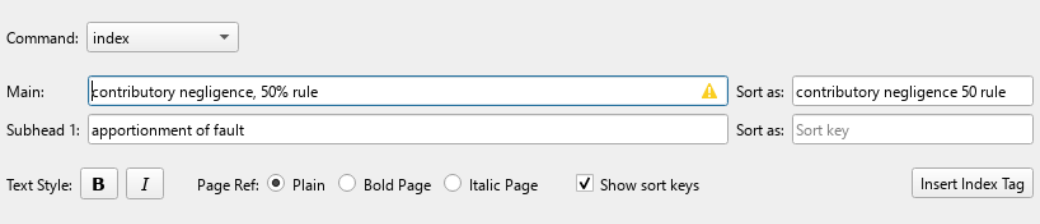

Figure 19. A syntax warning on a heading level, with a clean level beside it.

Page Reference Style

Plain / **Bold Page** / **Italic Page** decide how this entry's page number prints, not how its heading prints. The convention it serves is a familiar one: roman for ordinary mentions, bold for the page carrying an illustration, italic for the principal discussion, so a reader scanning a long run of page numbers can pick out the one that matters. It applies to this occurrence only — the same heading indexed elsewhere is unaffected. Bold and Italic write `|textbf` and `|textit` — real LaTeX commands, so the compiled index has something to call.

The setting returns to **Plain** after every insertion, deliberately: a styled page number is the exception, and one left switched on would quietly style the next dozen entries. A page range can carry a page style too: select text, choose Bold or Italic, and both ends of the range are written in the form `makeindex` reads, `|(textbf` on the opening macro and `|)textbf` on the closing one, so the whole span prints styled rather than just one of its page numbers. The style can be changed afterwards from the **Page** column of the entry table; only the opening half of a range is listed there, and the closing half follows whatever you set, so the two ends never disagree.

Inserting the Entry

Insert Index Tag (`Ctrl+K`) writes the macro. What you get depends on the editor tab, not on the panel:

- **Cursor placed, nothing selected** — a point reference: one macro at the cursor.
- **A span of text selected** — a page range: an opening macro before the selection and a closing one after it, with your text untouched between them. Ranges have their own

section in the index tree chapter, and a dedicated checker in the Tools menu for when the two halves drift apart.

Afterwards the panel resets for the next entry: the fields clear, the Subhead levels fold away, **Page Ref** returns to **Plain**, the sort fields empty and go back to following their headings, and focus lands in **Main**. **Show sort keys** is the one thing that survives — it describes how you work, not the entry you just made.

Where the Entry Goes Next

The macro reaches your source immediately — written into the tab's buffer if the file is open, straight to disk if it is not — and the new entry appears in the index tree and the entry table at once, without a reload.

What has *not* happened yet is the database write. Everything the project database records about your index is held in memory and written in one batch when you save, so `Ctrl+S` is still what commits a session's work. Saving is covered at the end of the Getting Started chapter.

Chapter 6

The Index Tree

The **Index Tree** is the hierarchical, back-of-book index view of every entry in the project (every heading, sub-heading, and sub-sub-heading), each with its own bracketed reference numbers linking back to where it actually appears in the source text (Figure 20).

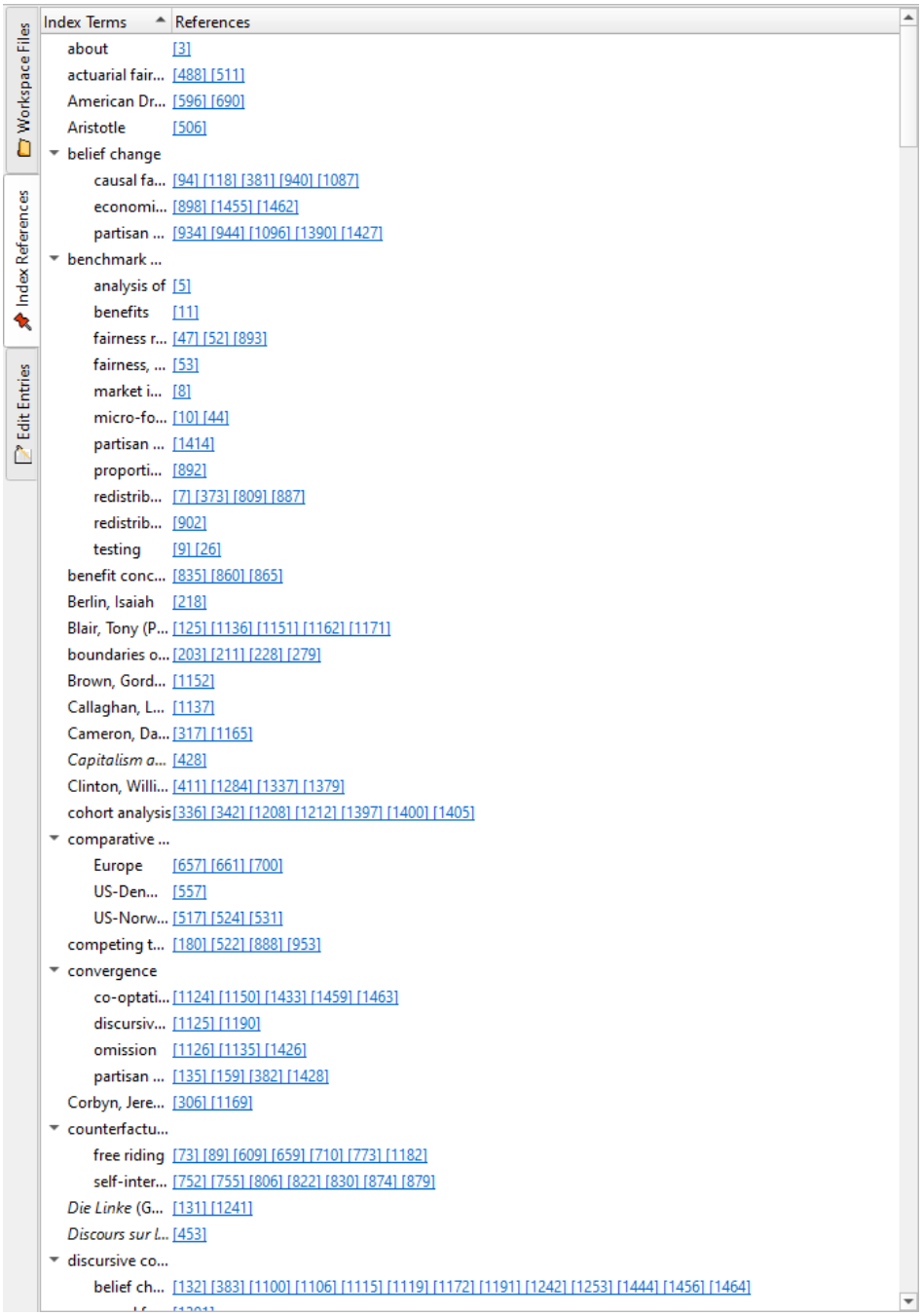

Figure 20. The Index Tree view.

It's the second of the three panels the left sidebar can show (**View → Focus Index Pane**, `Ctrl+Shift+I`), and for most day-to-day work (browsing what's already indexed, renaming a heading everywhere at once, deleting a term) it's the panel that allows high level editing of the index. The **Entry Table** is used to perform spreadsheet-style editing of single or multiple references and is covered in Chapter 7.

This chapter covers; reading and navigating the tree, how newly created entries appear in it, renaming and deleting entries, how range references are shown, and how legacy inline cross-references display.

Viewing and Navigating

The tree has two columns,

- **Index Terms** on the left showing the heading hierarchy itself, one row per main heading, sub-heading, or sub-sub-heading
- **References** on the right, showing each heading's own direct references as bracketed numbers, e.g. `[12] [45]`.

A heading with no reference of its own, a pure umbrella existing only to hold its sub-headings, has an empty References cell.

Sorting is alphabetical and case-insensitive, and it sorts by a heading's sort key rather than its displayed text.[11] The tree is fully expanded every time it loads or changes, so you're never hunting through collapsed branches for something that was just inserted. Collapse or expand any branch manually with the arrow beside it, the same as any standard tree view.

Clicking[12] a reference number in the **References** column jumps to that entry's exact location in its source file, opening the file into an editor tab if it isn't already open, moving the cursor there and highlighting the selected reference. This is usually the fastest way to find a specific `\index` macro in a large project without searching by hand.

Inserting Entries

New entries aren't created directly in the tree; that's the Index Entry window's job, which Chapter 5 covers in full. The tree reflects a new entry the moment it's inserted. As soon as you press Insert Index Tag (or `Ctrl+K`), the new heading appears here immediately, correctly sorted into place and expanded, whether it reuses an existing heading or creates a brand-new branch.

If the new entry's main heading, or a sub-heading along the way, doesn't exist yet, the tree creates whatever nodes are missing to hold it: inserting a first sub-heading under an already-existing main heading, for instance, adds just that one new row rather than rebuilding the branch. An existing node with the same heading text (matched case-insensitively) is always reused rather than duplicated.

---

[11] See Sort Keys vs. Displayed Text above for an explanation of this distinction.

[12] Reference numbers in the **References** column function as hyperlinks, visually indicated by the change of the cursor from an arrow to a pointing hand.

Editing and Deleting Entries

Double-clicking directly on a heading's text (at any level; main, sub-heading, or sub-sub-heading) opens it for inline editing in the tree; type the new text and press Enter (or click elsewhere) to commit. This is a full rename -- every reference filed under that node, including everything nested beneath it, is rewritten in its source file(s) to use the new heading text in one pass.

A rename is blocked, with a warning, if any of the affected references is mid-edit in the **Entry Table** (Chapter 7) at that moment, finish or cancel that edit first, then try the rename again. This prevents the tree's rewrite from silently overwriting an edit you haven't finished making somewhere else.

Right-click any node and choose **Delete Term "‹heading›"** to remove it and everything beneath it; every `\index` macro under that heading, at every depth, deleted from its source file. You're asked to confirm first, with an exact count of how many references will be removed; deleting an empty term (one with no references anywhere in its own subtree) skips the count and just asks to confirm the removal. This can't be undone once you've saved.

To delete just one reference rather than an entire term, use the **Entry Table** instead, deleting a heading's last remaining reference from there removes the now-empty node from this tree automatically.

Range References

A ranged `\index` entry reference is written as a matched opening/closing pair of `\index` macros in the source file, but only the opener it attached for display, the tree shows it exactly like a single reference, one bracketed number in the References column, with nothing to visually mark it as a range. The closing half of the pair never appears here at all.[13]

To see that a reference is actually a range, or to work with one directly, use the **Entry Table**. A range's opening reference shows an '`(`' in its Page column, and the Standard/Bold/Italic choice beside it sets the page style for the whole range. Only the opening half is listed; the closing half is updated to match, so the two ends never disagree. In the source that pairing is written `|(textbf` and `|)textbf`, the range marker first and the style after it. To check a project for range pairs that don't match up correctly, **Check Range Consistency...** is the dedicated tool for that.

Cross-References (See / See Also)

Every cross-reference in the project appears in this tree, filed under the heading it points from (Figure 21). How it appears depends on which of the two kinds it is.

Managed cross-references: anything created in the Cross-References tab (see *The Cross-References Tab*) shows as a child node reading *See Target* or *See also Target*, sorted ahead of that heading's real alphabetical sub-entries. Only the *See* or *See also* label is italicised; the target itself keeps whatever formatting its own entry gives it, so a case name that is italic in the index stays italic here. It is a read-only view: there is no reference number beside it, and clicking it does not go anywhere, because there is no `\index` macro in your source to go to. These live

[13] Technically, the closer of the range is present in the tree view, it's just held as a hidden field not displayed to the user. It's needed for when the user edits or deletes a ranged `\index` term.

in the generated `cross_refs.tex`, so to change or remove one, use the Cross-References tab. The tree updates as soon as you add, edit, or remove one, with no need to reload the project.

Legacy inline cross-references: a pointer written the old way, as a `see{Target}` or `seealso{Target}` suffix directly on an `\index` macro, is an ordinary index entry as far as the project is concerned. It sits wherever its macro sits and carries a normal, clickable reference number that jumps to it in the source. The difference between the two forms is real rather than cosmetic: one has a location in your files, and the other does not.

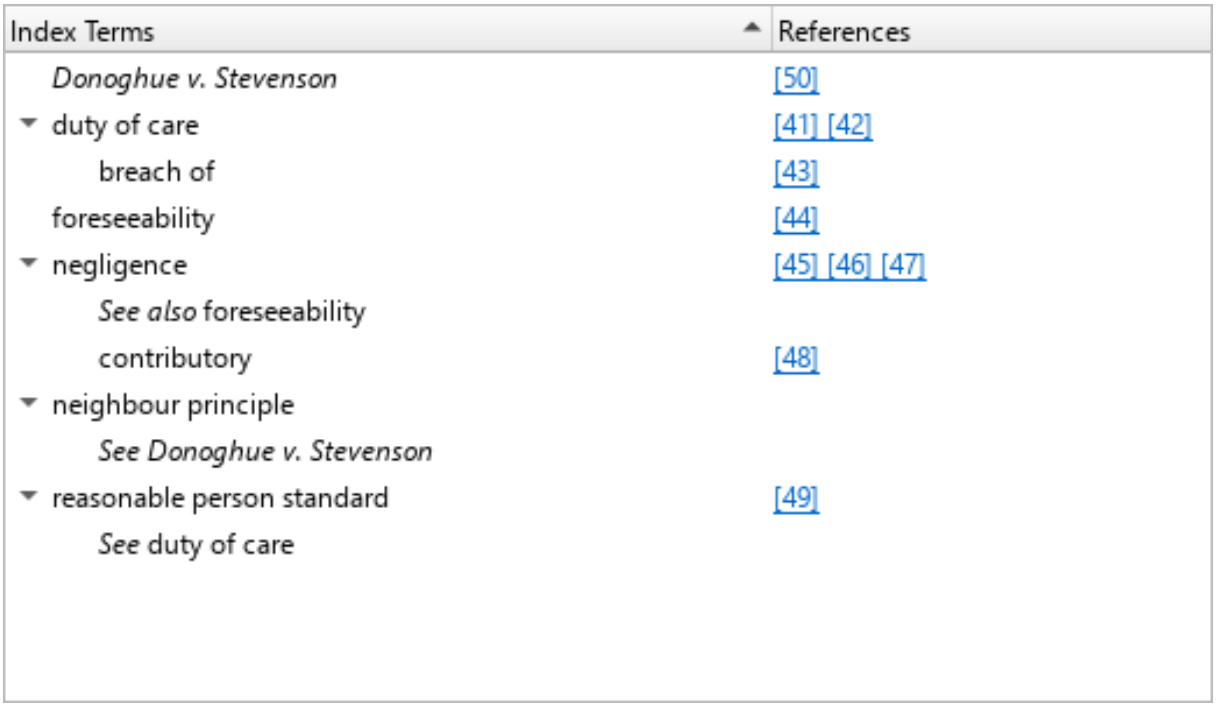

Figure 21. Cross-references in the Index Tree.

Migrating a legacy cross-reference does not remove it from the tree. It changes which of the two forms it takes, from an ordinary clickable entry to the read-only node. See *Migrate legacy cross-references*.

Chapter 7

The Entry Table

The **Entry Table** is the spreadsheet-style alternative to the **Index Tree** (Chapter 6), one row per reference rather than one row per heading, with every part of a heading broken out into its own column. It's the third of the left sidebar's panels (**View** → **Focus Edit Entries Pane**, `Ctrl+E`), and is composed of two sub-tabs, **Index** (Figure 22) and **Cross-References**.

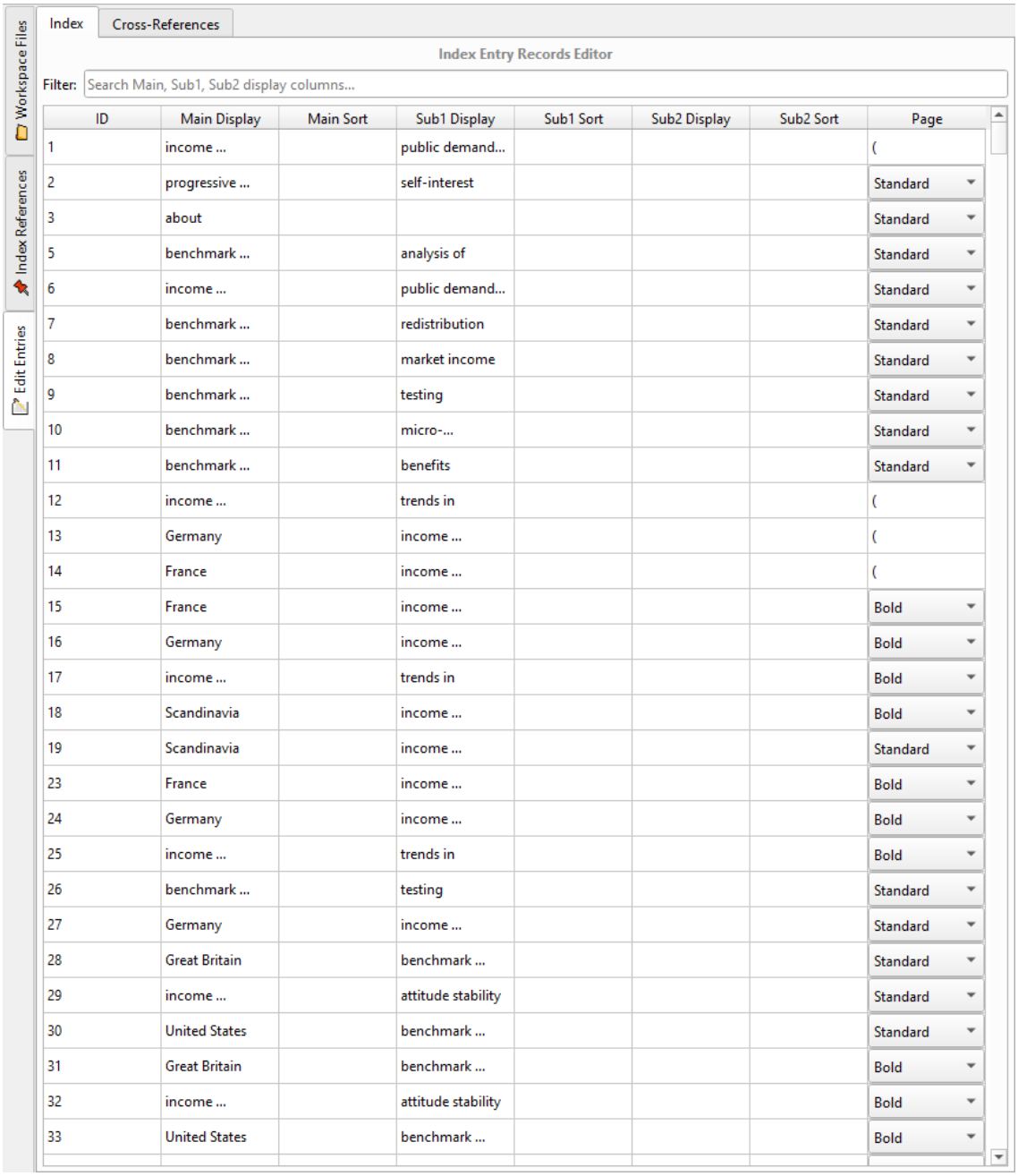

Figure 22. Edit entries index tab.

This chapter covers; the table's columns and inline editing, duplicating a reference to cross-post it under a second heading, and the dedicated **Cross-References** sub-tab for managing see and see also cross-references.

Editing Entries

Each row is one reference, laid out across eight columns,

- **ID:** a read-only copy of the unique identifier associated with each reference
- **Display/Sort pair:** one each for **Main**, **Sub1**, and **Sub2** (leave Sort blank unless the printed and alphabetized forms need to differ)
- **Page:** the page-style/range marker (standard, bold, italic, or a range parenthesis).

Any column can be hidden: right-click the header for a checklist of all eight (Figure 23). The Filter box above the table narrows the visible rows by searching the three Display columns as you type.

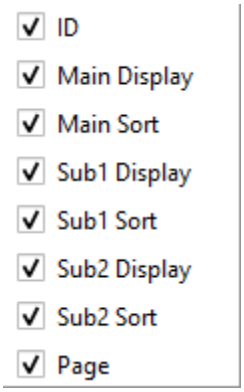

Figure 23. Column selection checklist.

Editing is per-cell but commits as a whole row: changing any column restages the entry's full heading from all of that row's current values, but nothing is written to the `.tex` source until you move off the row; to another row, or by pressing Enter. This is the mid-edit state that is guarded against in an **Index Tree** node rename (see Editing and Deleting Entries above).

An edit made here updates the **Index Tree** the moment it commits, the same as an edit made in the tree updates here, both views stay backed by the same underlying data, neither one is a separate copy.

Display and **Sort** cells carry the same warning icons as the **Index Entry** window's fields, on entries loaded from your files as much as on cells you have just edited — so an entry gets the same reading whichever way it was made. Hover a marked cell for what will happen to it. A cell reports but does not repair, there being nowhere in one to put a button, so the one-click correction stays in the **Index Entry** window.

Context menu editing

Contents of the **Entry Table** context menu.

Name inversion

Select **Invert name** to open the name inversion dialog, which allows the user to select a recommended inversion for a proper name. Name inversion is restricted to the content of the **Main** display field.

For a discussion of the name inversion dialog see Name Inversion below.

Delete reference

To remove a reference entirely, right-click the row (or a multi-row selection) and choose **Delete reference**. Unlike the **Index Tree** command **Delete Term**, this only removes the specific reference(s) selected, never a whole heading and its descendants.

Duplicate references

Select one or more rows, right-click, and choose **Duplicate references**. Each selected reference is copied; the exact same macro text, at the same page, and spliced into its source file immediately after the original, with its own fresh reference ID. The status bar confirms how many were duplicated.

The copy starts out identical to the original, including its heading, it's up to you to then edit the new row's **Main/Sub1/Sub2** display/sort columns to create the new index entry. A range reference duplicates as a linked opener/closer pair rather than a single orphaned marker, so the copy is still a valid, balanced range.[14]

Invert headings

Select **Invert headings** to invert the contents of the **Main** and **Sub1** fields in a reference.

The Cross-References Tab

This sub-tab is the current, dedicated way to manage a project's cross-references (Figure 24). Every *see* and *see also* pointer is its own row in its own list. This tab is the only place a cross-reference can be created or edited; the Index Tree shows them too, but read-only (see *Cross-References (See / See Also)*). The tab is composed of three columns,

- **Source**: the heading the cross-reference is attached to
- **Type**: the type of cross-reference (either see or see also)
- **Cross-Ref**: the target a cross-reference points a reader toward

[14] Only the new opener is displayed for the new entry, but both an opener and closer are written to file.

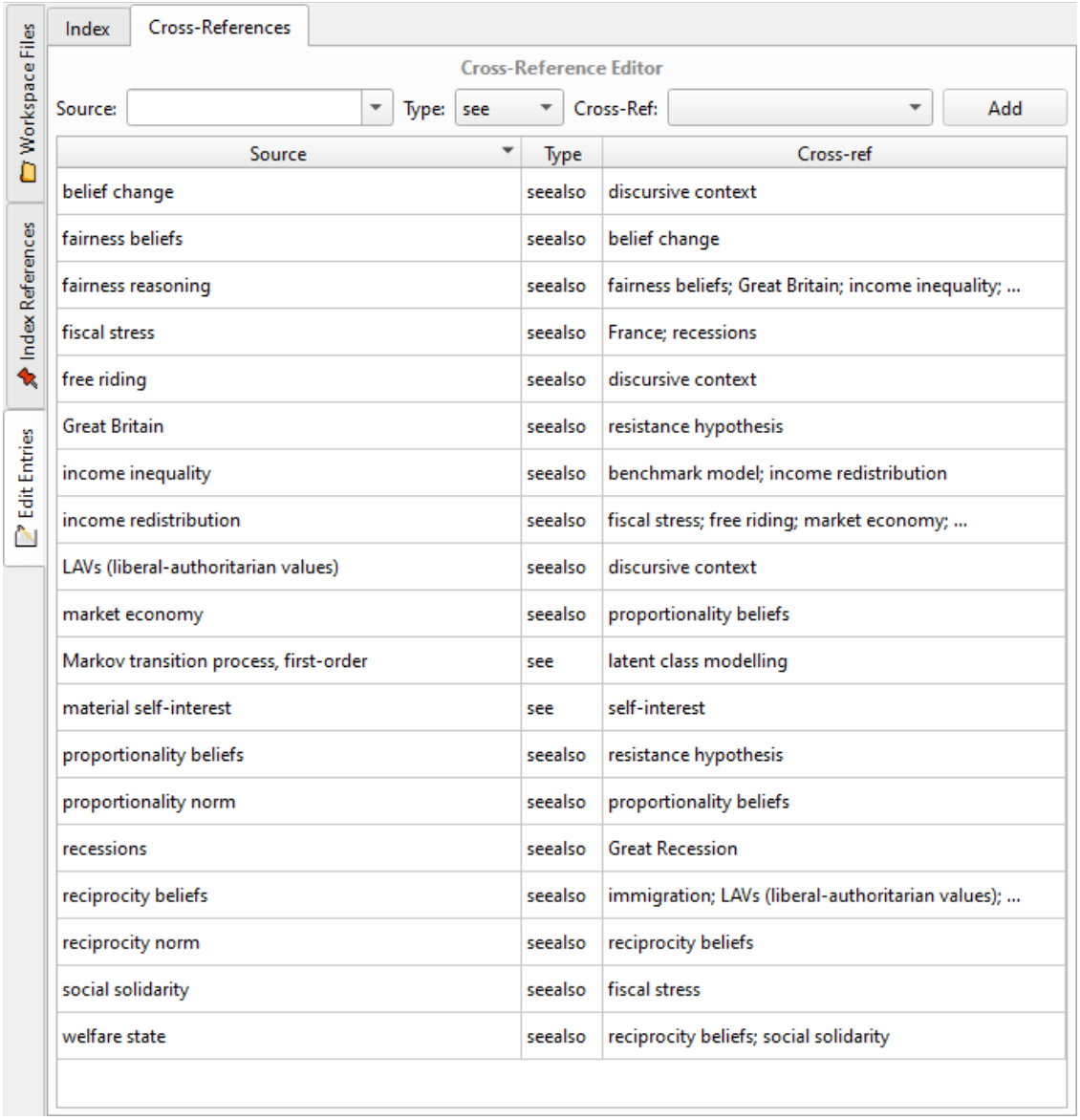

Figure 24. Edit Entries Cross-References tab.

Source behaves differently depending on Type, because the two mean different things: a *see also* supplements a heading that already has its own real references, so its Source is a dropdown of existing main headings only. A *see* is a pure redirect with no references of its own (for example, “material self-interest” pointing to “self-interest”), so its **Source** field is freely typeable instead, letting you name a heading that doesn't exist anywhere else in the index.

To create a new cross-reference, fill in **Source**, **Type**, and **Cross-Ref**, then click the **Add** button.

Existing rows can be edited directly in the table, **Type** is a dropdown, **Source** and **Cross-Ref** are plain text, and can be removed via the right-click menu (via **Remove Selected Cross-Reference(s)**) or the Delete key on a selection.

Source file independence

None of this touches the `.tex` chapter files. Every change here regenerates the project file `cross_refs.tex`, from scratch.

The first time a project has any cross-references, use **Tools → Insert Cross-References File...** once to splice `\input{cross_refs.tex}` into the base file. After that every further add, edit, or removal updates that file's contents automatically.

The user should not edit `cross_refs.tex`, as this file is generated automatically by the application. Any user edits to this file will be lost the next time it is generated. All management of cross-references should happen through the **Cross-References** tab.

If a project has cross-references written the old inline way from before this tab existed, they won't appear here automatically, **Tools → Migrate Legacy Cross-References...** finds them and offers to move them into this system.

Chapter 8

Custom LaTeX Commands

Some LaTeX projects don't call `\index` directly, they wrap it in a project-specific macro instead, often to bundle in extra formatting or logic around every indexing call (something like `\isidx{term}`, expanding to `\index{term}` plus, say, automatic italics). This chapter covers teaching the editor about a command like that, creating one, and adopting it into a specific project so it appears as a choice in the Index Entry window.[15]

None of this is required, plain `\index` always works, with or without any custom commands defined. This is purely additive, for projects that already use a wrapper macro or want to start using one.

Creating a command

Tools → Create LaTeX Command... (`Ctrl+Alt+C`) opens a simple two-field dialog (Figure 25) **Command name** and **Command definition**. Enter the exact `\newcommand`, `\renewcommand`, `\providecommand`, or `\def` declaration as it would appear in your LaTeX preamble into the definition field. **Save** stores it; there's nothing project-specific about this step yet, it's saved to the application's global library of commands, available to every project.

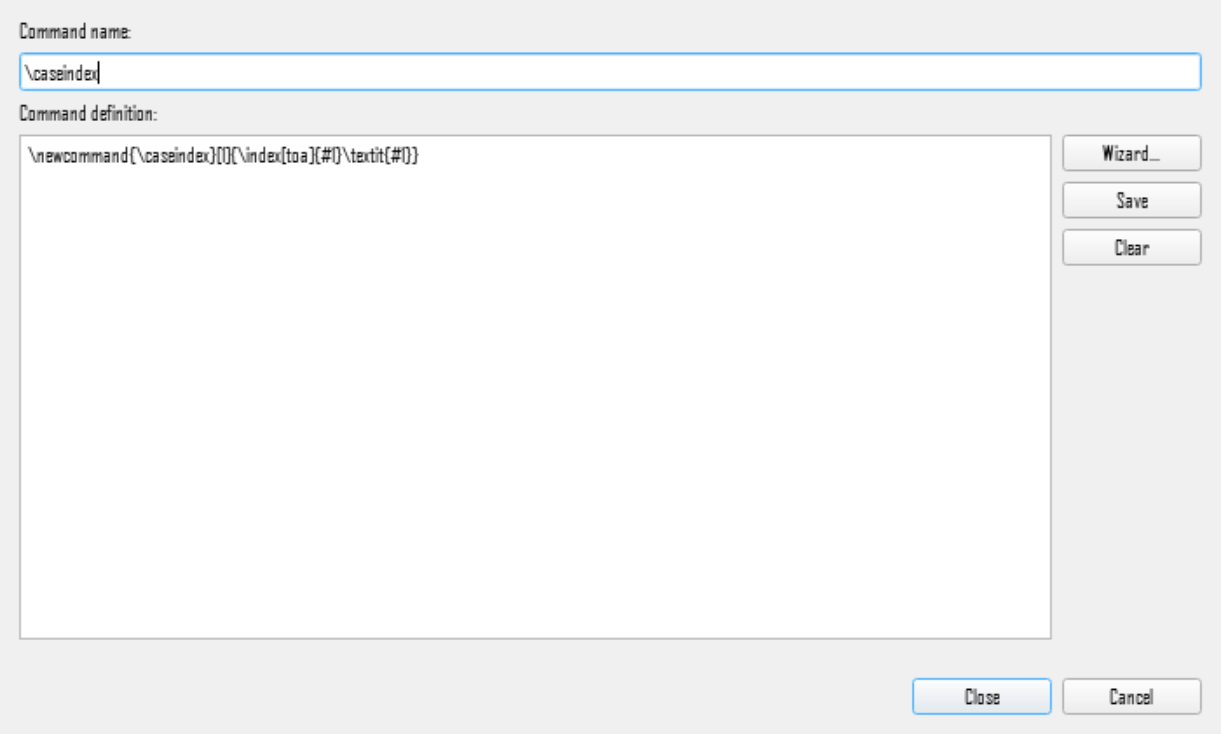

Figure 25. Custom LaTeX command dialog.

[15] Using this feature requires the user to be conversant with how to use `\newcommand` and `\def`. If you are not, I'd suggest first looking at the Overleaf page on commands and then at the documentation for LaTeX2e (section 12.1).

If you'd rather not write the declaration by hand, **Wizard...** (Figure 26) walks through it in two steps instead: choosing `\newcommand` or `\def`, the command's name, how many arguments it takes, and an optional default (`\newcommand` only); then the replacement text itself on the second page (using `#1, #2, etc.` for the number of arguments specified). Finishing the wizard fills the Name and Command definition fields back in for you to review and edit, before saving.

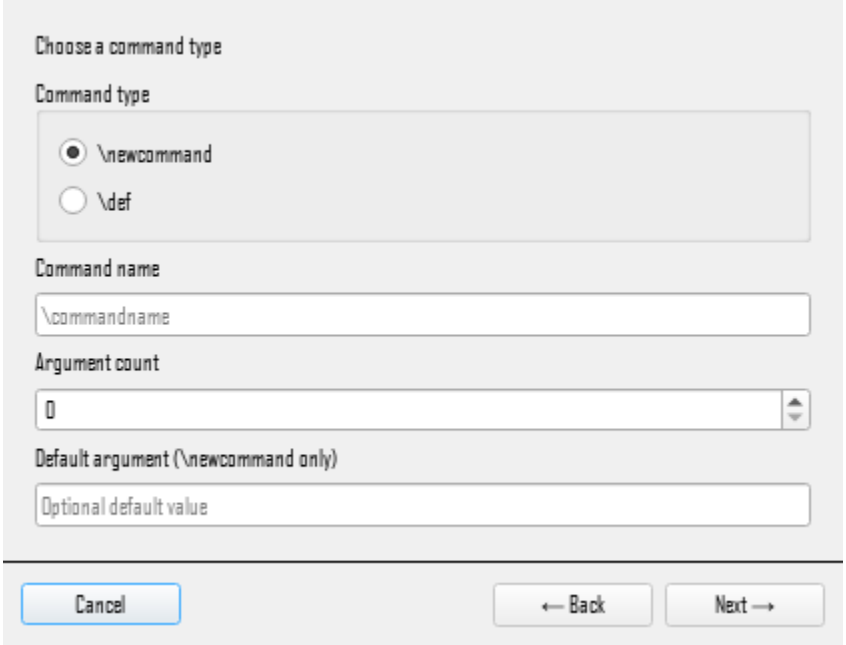

Figure 26. Custom command wizard.

For a command to actually show up as a choice in the **Index Entry** window, its definition has to be a `\newcommand / \renewcommand / \providecommand`, not a plain `\def`, and its body has to call `\index` somewhere inside it. A helper macro that does something else entirely (say, a custom footnote formatter) can still be created and saved here, but it will never appear as an indexing option, since the editor has no way to know it's meant for indexing.

Managing project commands

Tools → **Manage Project Commands...** (`Ctrl+Alt+M`) is where a custom command actually gets attached to a specific project (Figure 27).

Available Commands on the left is the global library of custom commands; everything ever saved via the **Create LaTeX Command...** command. **Commands in Project** on the right is the project's adopted subset. Select one on the left and clicking **Add** → moves it into this project; select one on the right and ← **Remove** takes it back out.

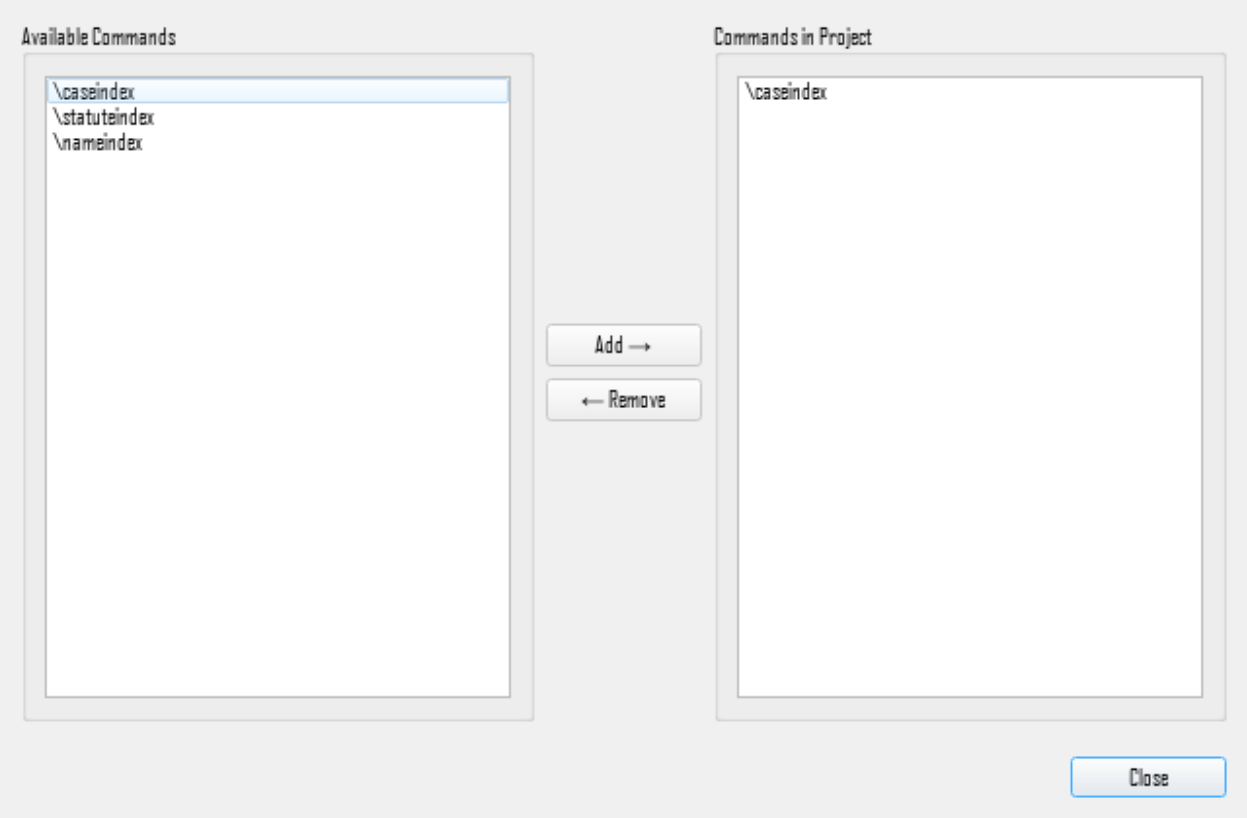

Figure 27. Custom command management dialog.

Only commands adopted into the current project appear in the **Index Entry** window's Command dropdown; everything else stays available here but out of that list. Adopting or removing a command takes effect immediately, without needing to reopen the project, and the choice is remembered the next time this project is opened.

Once adopted, choosing that command from the Index Entry window's dropdown before pressing **Insert Index Tag** writes an entry using that command's name instead of plain `\index`, choosing `isidx` there, for instance, produces `\isidx{term}` rather than `\index{term}` at your cursor.

Chapter 9

Tools Menu

This chapter works through the Tools menu one action at a time. The following are covered,

- Head Notes
- RTF Export
- Index Statistics
- Range Consistency Check
- Resyncing Index Data from Disk
- Managing Pruned Files

A handful of other tools actions are covered where they're more directly relevant, rather than repeated here,

- Resync Workspace Files from Disk (Chapter 4)
- Migrate Legacy Cross-References (Chapter 6).
- Create LaTeX Command and Manage Project Commands (Chapter 8)

Head notes

Tools → Create Head Note... (`Ctrl+Shift+H`) adds introductory text that prints at the very start of the index, before its first entry. A short paragraph explaining how the index is organized, a note on conventions used, or similar front matter specific to the index itself rather than the book as a whole (Figure 28). This tool requires a **base file** be defined for the project, since both the note's storage and its injection target depend on knowing both the project and file.

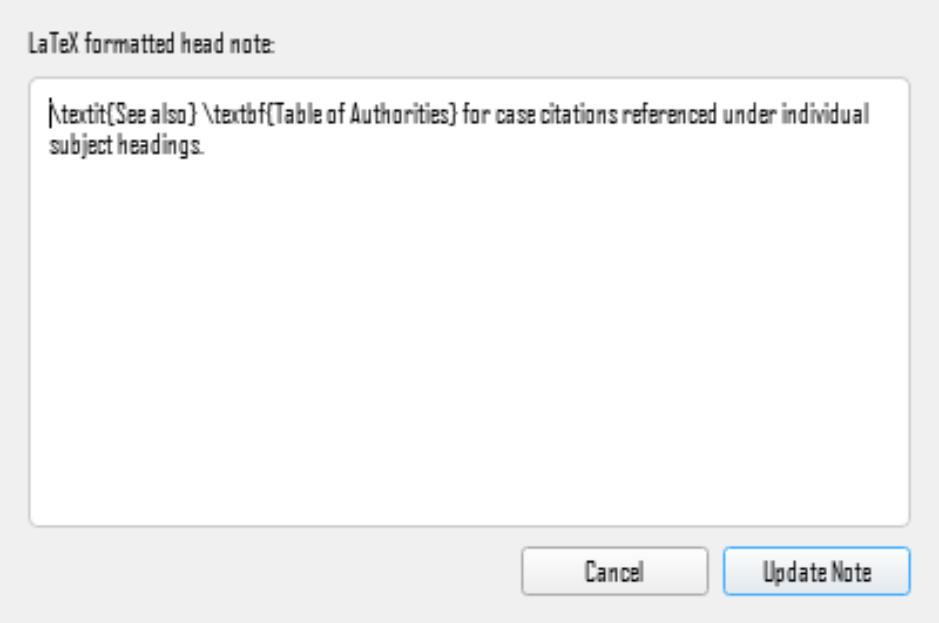

Figure 28. Create head note dialog.

Enter the note's text (raw LaTeX is fine, the same as any other display-text field in the application) and accept (**Update Note**). It's stored per-project and spliced into the base document as `\indexprologue{...}`, `imakeidx's` own mechanism for text appearing immediately before the printed index, positioned right before whatever command actually prints it.

If a project already has a head note, the dialog opens pre-filled with the existing text for editing rather than starting blank; accepting again replaces the old note in place — the previous `\indexprologue{...}` block is found and removed before the new one is inserted, so re-running this never leaves two behind.

RTF export

Tools → **Create Rtf File** (`Ctrl+Alt+R`) compiles the project's index and exports it as an RTF file, for review outside the application or for providing a draft index to the author (Figure 29). This tool requires a **base file** and both a LaTeX compiler and an index engine executable configured in Preferences → LaTeX. You're warned by name if either is missing rather than failing partway through.

Figure 29. RTF viewer.

The pipeline behind it is the same partial LaTeX one described in Chapter 2.

1. Run a single `pdflatex`[16] pass in draft mode. No PDF is actually produced, just the `.aux` LaTeX needs to resolve every `\index` call

[16] While `pdflatex` is named here as the LaTeX compiler, one of three compilers can be selected by the user. See LaTeX Compiler below for a description of the available options.

2. generates a fresh `.idx`; `makeindex` or `xindy` sorts it into an `.ind`
3. That `.ind` is parsed and rendered as an `RTF` file.

A progress dialog tracks which stage is running, including live page-count feedback during the compile step, since a large project's export can take a while.

Because the export compiles the real `.tex` files on disk, save before running it — entries inserted since the last save are still in the editor's tab buffers, and won't appear in the compiled index.

The result is saved as `<Project Name>_index.rtf` in the project's output `build` folder. If **Display RTF file on creation** is checked (Preferences → RTF), the read-only RTF preview opens automatically once export finishes; otherwise you'd open the exported file yourself.

Index statistics

Tools → **Index Statistics...** opens a small read-only summary of the current project's index (Figure 30). **Main** headings, **Sub1** headings, and **Sub2** headings are counts of distinct headings at each of the three levels; **Total index references** is every actual page reference in the project, at any depth; **Total cross-references** is a separate count, scoped specifically to inline `see/seealso` markers found while scanning your regular `.tex` files, it does not include rows managed by the dedicated **Cross-References** tab, since that tab's `cross_refs.tex` file is deliberately excluded from index scanning. The dialog reads straight from the project database, so its numbers describe the project as last saved rather than what is currently on screen — save first for a count that includes this session's work.

The dialog shows one count per heading level, and the number of levels is the index format's own ceiling rather than a fixed three. LaTeX allows three, so three is what you see here; the same dialog in the Word index editor shows three and in InDesign four, because those formats say so. It is the format that decides, not the dialog.

Three details of the counting are worth knowing, because each of them looks wrong for a moment and is not. A heading at a level is a distinct path down to that level rather than a distinct word, so Kant:reception and Hume:reception are two headings at the second level and not one. A level is counted as it is stored, sort key and all, so a heading filed under a sort key and the same heading without one count as two, which is what the index tree already shows and what Check Index exists to report. And the closing half of a page range is not counted as a reference of its own, because the opener already accounts for it.

The counts cover the whole project. A project carrying more than one named index is counted together rather than index by index.

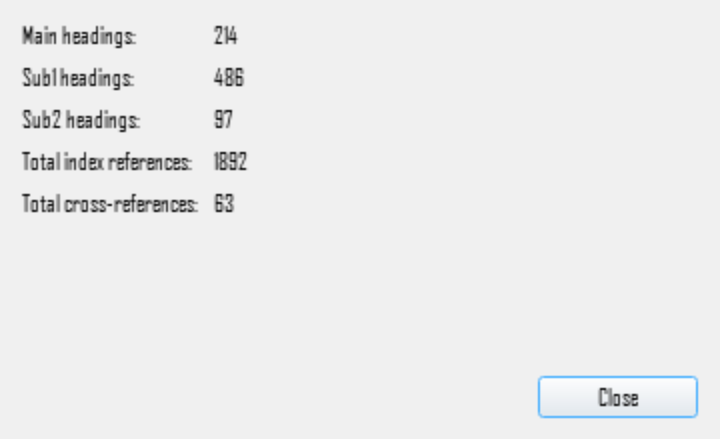

Figure 30. Index summary dialog.

Range consistency check

Tools → Check Range Consistency... scans every page-range reference in the project for pairing problems that external editing of `.tex` files or externally run scripts can leave behind (Figure 31). Results are grouped into three categories, each pre-checked in the results list, and **Apply Selected** acts on whichever ones you leave checked.

Three types of consistency checks are made,

- **Malformed ranges** is an opener with no closer, or a closer with no opener, the fix deletes the stray, unmatched marker itself.
- **Overlapping ranges** is a second range for the same heading starting before the first one has closed, the fix merges them into a single continuous range, spanning from the first range's start to the second range's end.
- **Enclosed point references** is a plain, unstyled single-page reference that falls inside an already-open range for the same heading, since the range already covers that page, the fix removes the redundant standalone point. A bold- or italic-styled point falling inside a range is left alone since that's a deliberate indexing convention.

Nothing is changed until you review the list and click **Apply Selected**; uncheck anything you'd rather handle by hand first, or Select None and just use the report to see where the problems are.

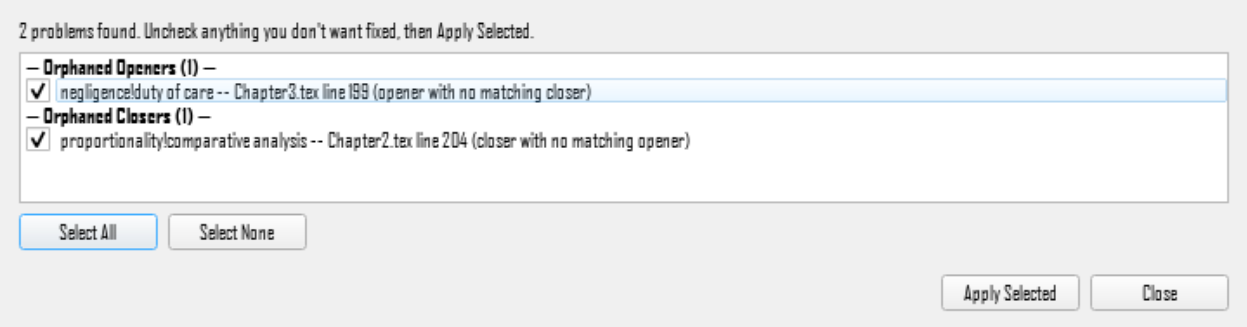

Figure 31. Range consistency check dialog.

Resyncing index data from disk

Tools → **Resync Index Data from Disk** fully re-scans every tracked `.tex` file in the project and rebuilds the **Index Tree** and **Entry Table** from what's actually in them right now, the heavier counterpart to **Resync Workspace Files from Disk**, which only reconciles the file list, not index content.

This is also what runs, with your confirmation, whenever the editor detects that a tracked file's content has changed since the project was last open. A Files Changed dialog is displayed on project open, naming which files, and asking if they files should be resynced (Figure 32).

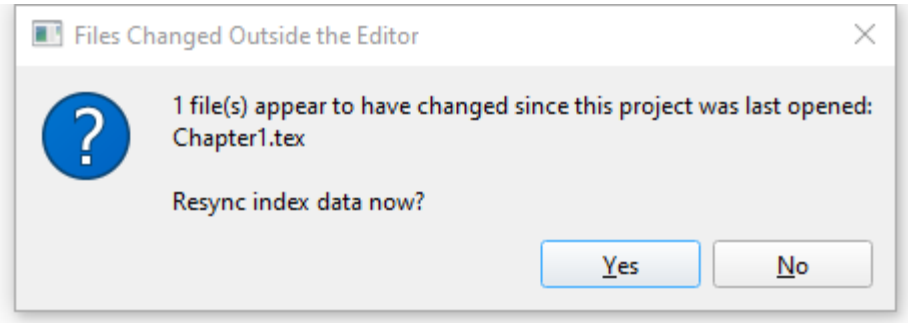

Figure 32. Files changed dialog.

The manual Tools action is the same operation, available any time rather than only at that moment.[17]

This is a real rebuild, not an incremental update: every reference's internal ID is reassigned from scratch. Anything tracked by a stale ID — an entry mid-edit in the Entry Table, an undo/redo step queued from before the resync — doesn't survive it, which is why the editor only ever runs this automatically when nothing unsaved is at stake, and otherwise leaves it to you to run deliberately, with that trade-off understood.

Unsaved index changes fall in the same category: they exist only in memory, and a resync rebuilds purely from the files. The manual Tools action therefore asks before it proceeds if anything is unsaved. **Save** writes everything out first, so the rebuild reads your changes straight back in from the files and nothing is lost; **Discard** rebuilds anyway and abandons them; **Cancel** leaves the project exactly as it was. The automatic resync is more cautious still, refusing to run at all while anything is unsaved.

Managing pruned files

Tools → **Manage Pruned Files...** is a checklist of every currently pruned file in the project, letting you bring one or many back into active scope at once (Figure 33). See Pruning Files and Restoring Pruned Files above for a discussion of pruning and restoring.

[17] The application has a file modification watch engine, so while it is open it detects if one of the project files is modified by an external app and resyncs.

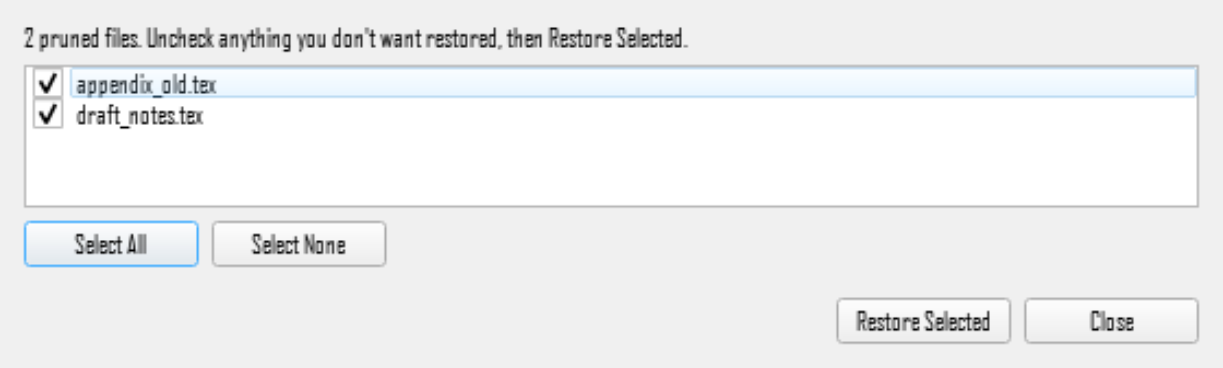

Figure 33. Manage pruned files dialog.,

Migrate legacy cross-references

Selecting **Tools → Migrate Legacy Cross-References...** launches the cross-reference migration dialog (Figure 34). The dialog lists every cross-reference written inline in your source files, with its heading, file and line, and its see or see also target. Every item starts selected; clear anything you would rather leave alone, then click **Migrate Selected**. Each one is removed from its original `.tex` file and added to the Cross-References tab, and so to `cross_refs.tex`, instead. Migrated cross-references remain visible in the Index Tree, in the read-only form described in *Cross-References (See / See Also)*.

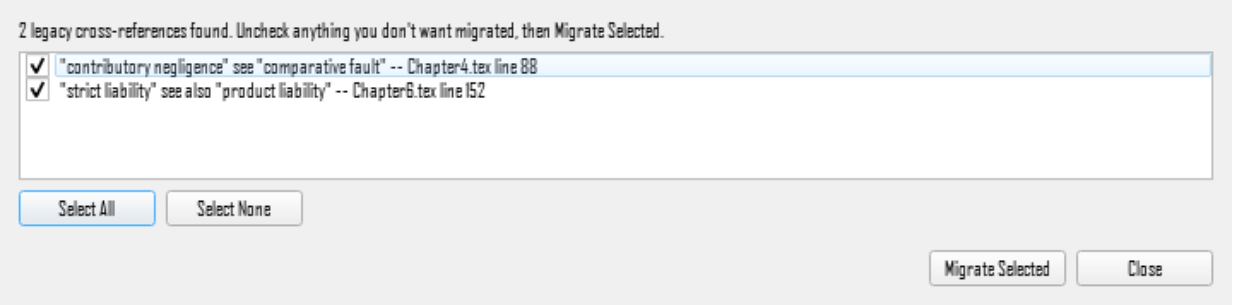

Figure 34. Migrate legacy cross-references dialog.

You don't have to go looking for this. When a project containing inline cross-references is opened, the editor offers to review them (Figure 35). Choosing **Yes** opens the same dialog. Choosing **No** won't ask again for that project, and the Tools action stays available whenever you want it.

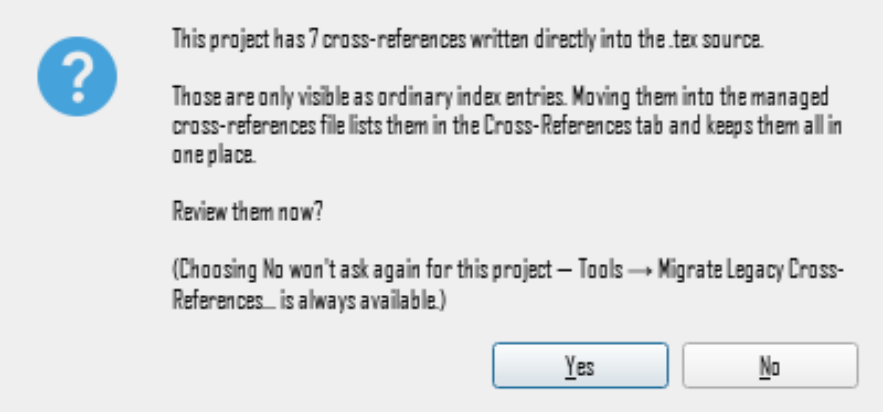

Figure 35. Prompt on project open for legacy cross-references.

A base document is required: migrating removes each pointer from your source files, and `cross_refs.tex` only reaches the compiled index through the `\input` line that goes into the base document (see *The Base File*). Migration therefore adds that line as part of the same step, and refuses to run at all if there is nowhere to put it. Nothing is removed from your files unless the link can be made, so a project without a base document is left exactly as it was. If you see a message to that effect, set a base document and run the migration again.

Building a Table of Authorities

A Table of Authorities is the separate index of cases, statutes and other authorities that a legal book carries alongside its subject index. Tools → Build Table of Authorities... reads the citations your project already contains, groups them into the sections a table of authorities uses, and shows you the table it would build before anything is written.

The tool parses citations rather than matching them against a list, which is why it recognises forms it has never seen: a citation is a formal language, and the same reading that finds Roe v. Wade, 410 U.S. 113 (1973) finds the case reported beside it. Three citation systems are understood, Bluebook for United States material, McGill for Canadian and OSCOLA for British, and which one applies is a setting rather than a guess.

Nothing is written until you accept it. Review the proposed table, and where an entry is wrong, correct the citation in the index rather than the table: the table is built from the index and will follow. The Check Index rules for authorities, all off by default in an ordinary index, are worth turning on while you work on one.

The in-app help topic Table of Authorities carries the detail, including what each section holds and how a short form such as ibid is resolved back to the authority it refers to.

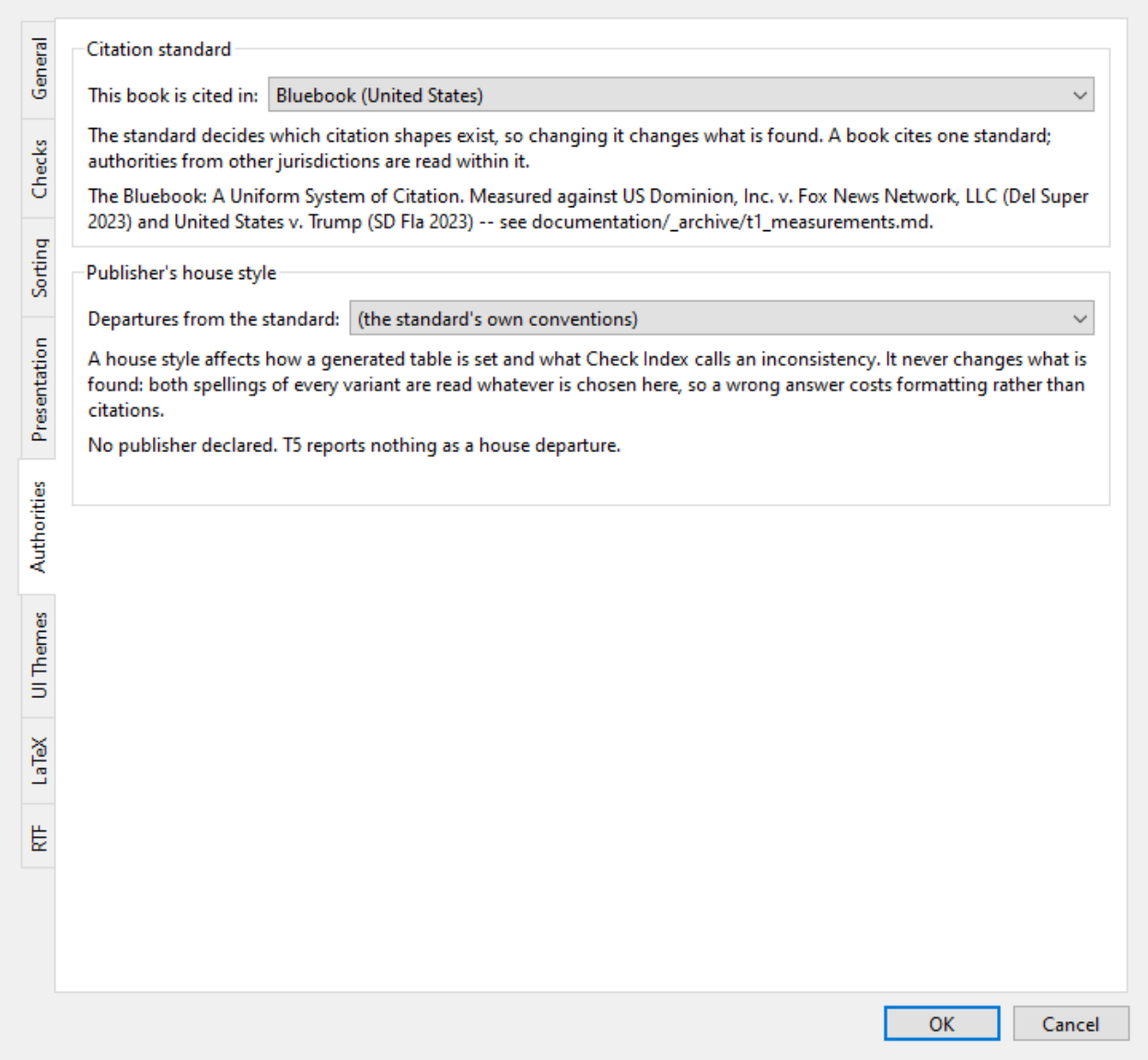

Figure 36. Authorities preferences: the standard the book is cited in.

Check Index

Tools → Check Index... reads the whole index and reports what looks wrong with it: two headings that differ only by a plural, a see reference pointing at a heading that no longer exists, a run of forty undifferentiated page numbers under one term. It is the tool to run before handing an index over.

Forty-five rules in five groups, of which thirty-three are on by default. The basic group looks at punctuation, spacing and characters LaTeX treats specially; the headings group finds the most on a real index, comparing headings that read alike but file apart; the cross-reference group follows every see and see also to its target; the locators group looks at page numbers and ranges; and the authorities group, off by default, applies only to a Table of Authorities.

It never changes anything. Most of what it finds has no mechanical repair, because being told that two headings disagree does not say which of them is right, so it reports and you correct in

the index tree or the entry table where you have validation and undo. For the faults that can be repaired mechanically, see Repair Index Entries below.

Which rules run is set in Preferences → Checks, per project once a project is open, so an index of statutes and an index of concepts need not run the same set. The check reads the database rather than your open buffers, so save first if you have been editing.

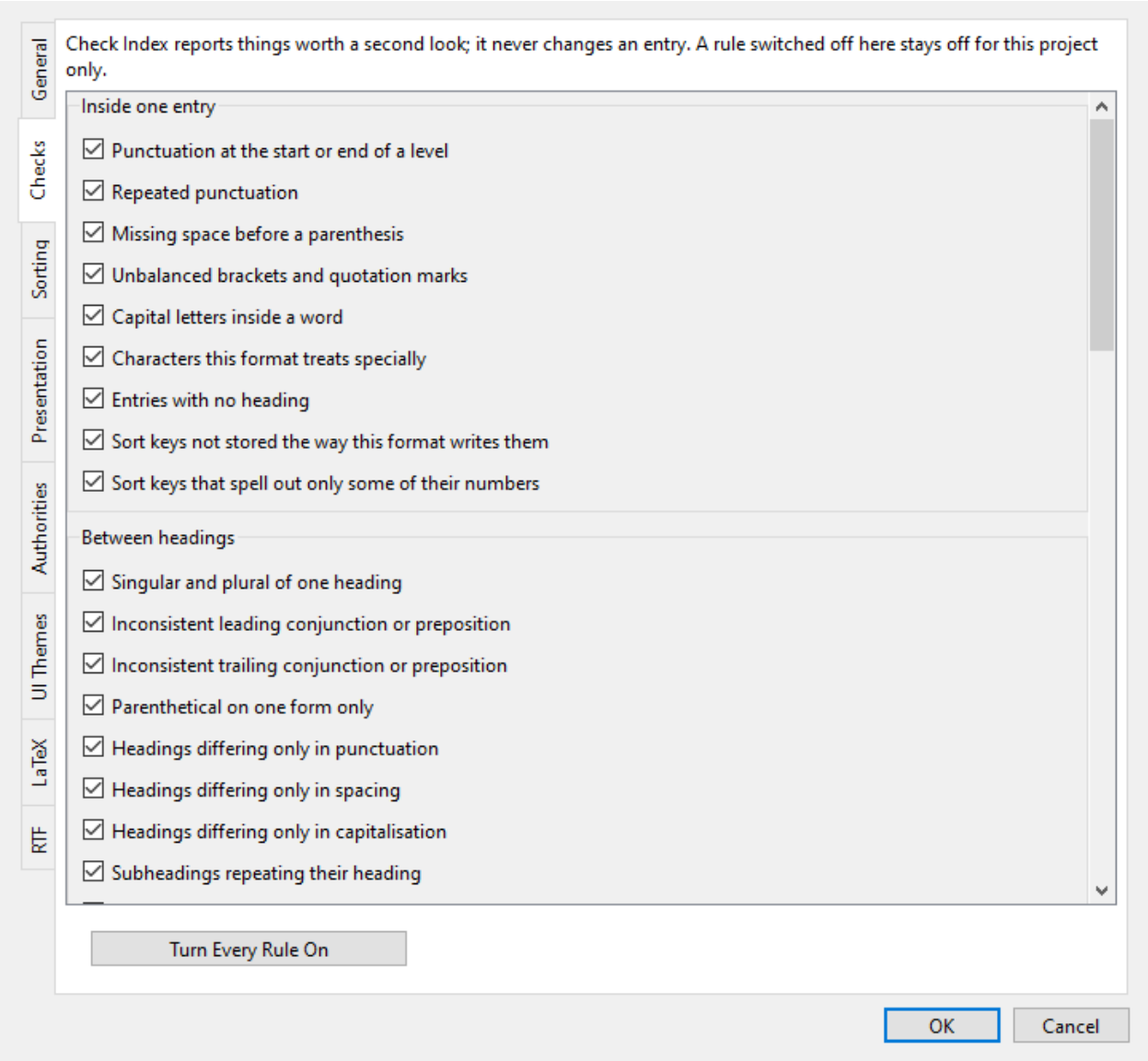

Figure 37. Checks preferences: which rules run for this project.

Repair Index Entries

Tools → Repair Index Entries... finds entries whose markup the index engine will misread, proposes a fix for each, and applies the ones you approve as a single undoable change. The entry window has been able to make the same repair since it was written, but one field at a time; this is the same repair offered across the whole index at once.

It will only make repairs the syntax checker already knows how to make, and only the ones it can make mechanically. A problem with no known fix is passed over rather than guessed at.

It never changes what a heading says. Every repair is about characters the index engine will misread, and none is about wording, spelling or filing. If a heading is wrong rather than malformed, Check Index is the tool that will point at it and you are the one who decides.

Every row starts approved; uncheck anything you would rather leave, then apply. The whole approved set lands as one command, so a single undo reverses all of it. An empty list means nothing in the index carries a fault this tool knows how to fix, which is a real answer and not a failure to look.

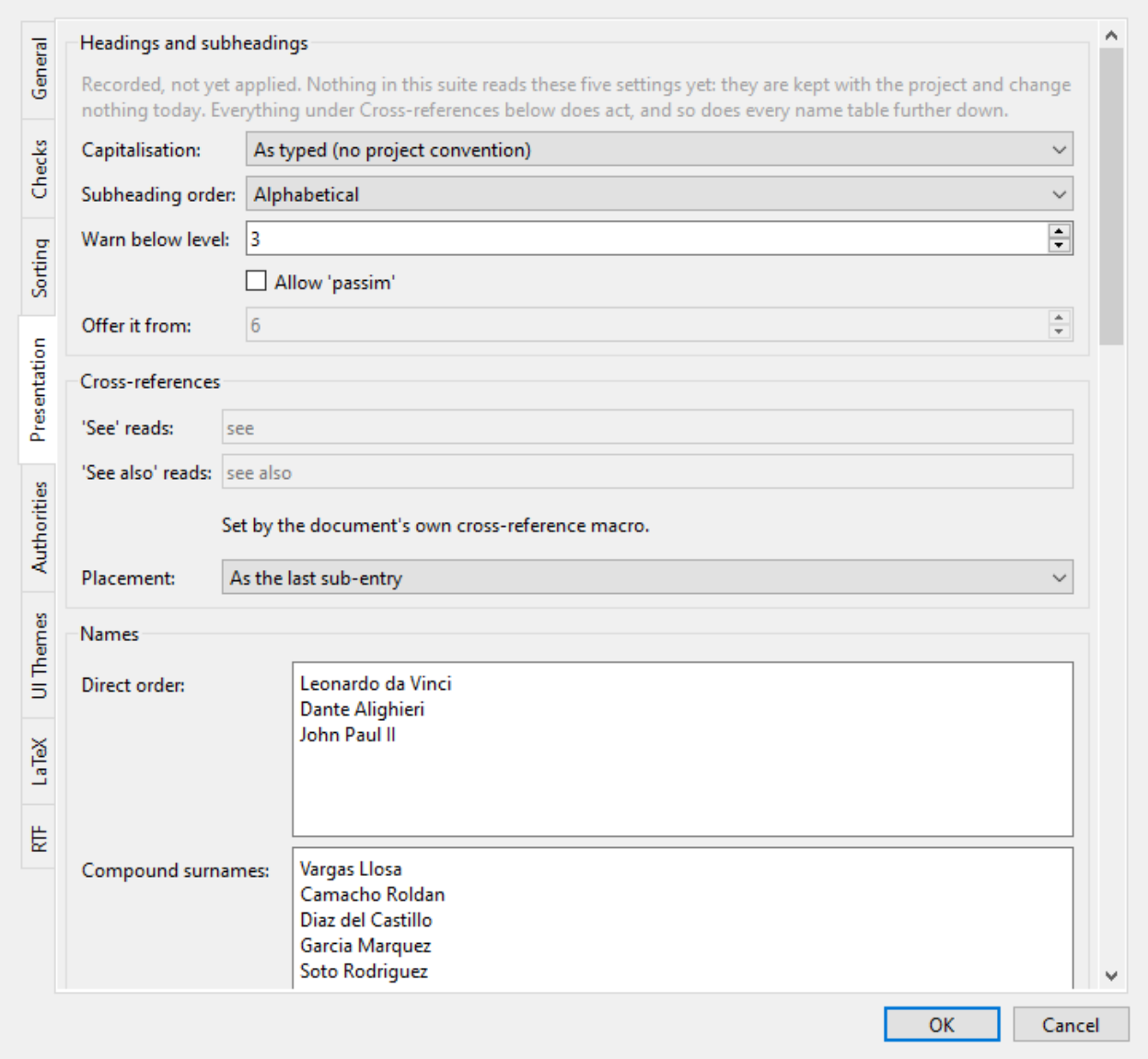

Figure 38. Presentation preferences, which decide how an entry reads on screen.

Chapter 10

Additional Features

This chapter covers two features that support the core indexing workflow.

Advanced Search

Edit → **Advanced Search...** (`Ctrl+Shift+F`) opens a separate, non-modal search window (Figure 39). Unlike **Find** (`Ctrl+F`), which searches only the file open in its editor tab, this searches all active (unpruned) files in the project at once, independent of what's open in a tab.

Two search modes are available, as separate sub-tabs: **Fuzzy Match Engine** and **Exact Subphrase Match**.

- **Exact** is a plain case-insensitive literal substring search, what you'd expect from Find, just project-wide.
- **Fuzzy** uses approximate string matching to find lines that are close to your search phrase without matching it exactly, useful for a term you're not sure was spelled or hyphenated consistently, governed by a Minimum Levenshtein Similarity slider (1-100%, default 75%). Raise it for stricter matches, lower it to catch looser variants at the cost of more noise.

Results are grouped by file, with each match showing its line number, and, in fuzzy mode, its match score, alongside a text snippet. Double-clicking a result opens that file into an editor tab, if it isn't already, and jumps straight to the matching line — the whole line is highlighted, rather than the more precise macro-boundary jump the **Index Tree** and **Entry Table** use since a search hit can land on any ordinary prose, not just inside an `\index` macro.

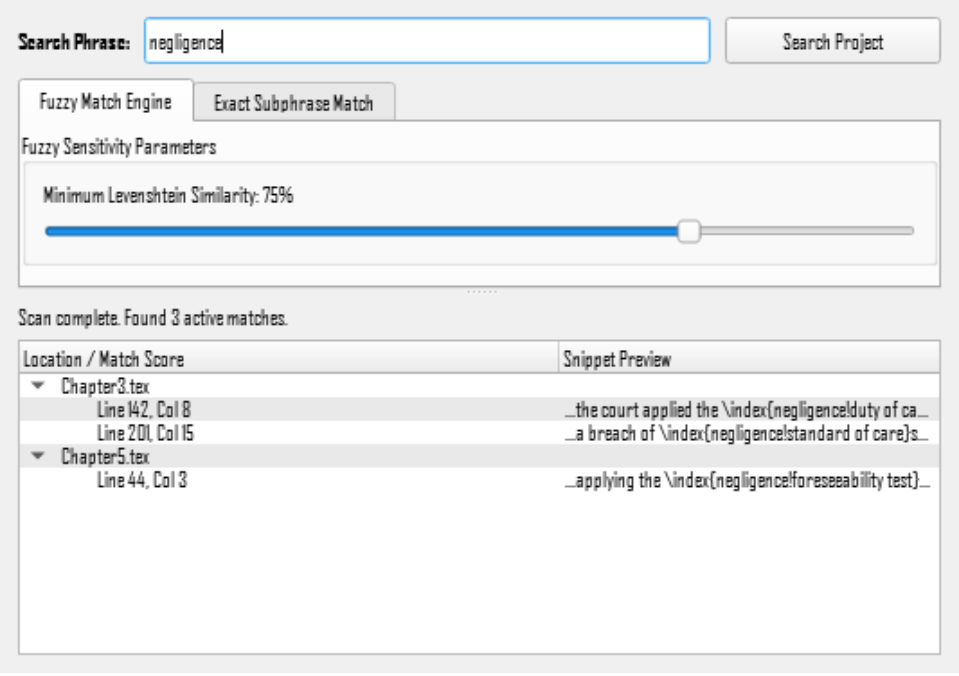

Figure 39. Advanced search dialog.

Name Inversion

Indexes alphabetize personal names by surname (“Winston Churchill” needs to be filed, and displayed, as “Churchill, Winston”). Name Inversion does that conversion for you, handling the harder cases (particles, compound surnames, generational suffixes) rather than requiring you to work out the correct inverted form by hand every time.

In order to activate, right-click a row in the Entry Table and choose **Invert name** (Figure 40).[18]

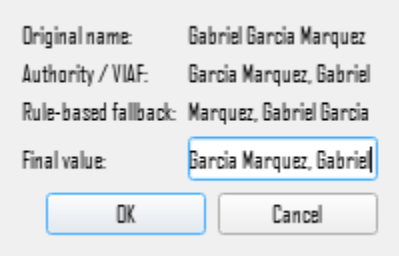

Figure 40. Name inversion dialog.

Every suggestion comes from up to two independent sources,

1. A rule-based conversion, worked out locally from the structure of the name itself (e.g. `Ludwig van Beethoven → van Beethoven, Ludwig`, or `Abdel Fattah el-Sisi → el-Sisi, Abdel Fattah`), which is always available and requires no network access.
2. An authority-record lookup against VIAF (the Virtual International Authority File, an international library service aggregating official name-authority records) and the Library of Congress. The authority lookup is generally more reliable for well-known names, since it reflects how libraries themselves catalogue that person, but needs a network connection and can take a moment the first time a given name is looked up; after that, the result is cached locally, so a repeat lookup of the same name is instant.

Nothing is applied silently, a dialog always appears first, showing the original name alongside both suggestions (when an authority match was found; only the rule-based one otherwise). Click either to use it, or type a different value directly. **OK** writes the chosen text into the Main heading, the same as any other Entry Table edit, staged, then committed to the `.tex` source when you leave the row; **Cancel** leaves the entry untouched.

If you type a correction that matches neither suggestion, that correction is remembered locally against this exact name, the same name offers your corrected version first the next time, instead of repeating the same suggestion you already had to fix.[19]

[18] Name inversion always acts on the row's Main heading.

[19] If you change the content of the Final value field, an additional selection opens in the dialog, allowing you to chose from a dropdown of common reasons for a manual change or allowing you to enter a new reason. This information is saved in the application names database.

Chapter 11

Preferences

Edit → Preferences... (`Ctrl+,`) opens a single dialog covering everything in this chapter, laid out across eight tabs down the left-hand side. This chapter covers General, LaTeX and UI Themes; the RTF tab is covered in Chapter 9 (see RTF export above).

General

Settings that belong to the application rather than to any one project. Unlike everything else in this dialog, these are shared by every project you open, and each takes effect as soon as you accept the dialog (Figure 41).

Undo stack size is how many index operations `Ctrl+Z` can step back through, 200 by default. Lowering it discards the oldest steps straight away rather than waiting for your next edit, so the number shown is always the depth you actually have.

Recent Projects governs the File → Open Recent list. **Show recently opened projects on the File menu** turns the feature on and off; it is on by default, and switching it off hides the submenu entirely and stops new projects being recorded, while keeping whatever is already stored. **Projects to list** sets how many appear, from 1 to 25, with 10 the default — lowering it hides the oldest entries rather than deleting them, so raising it again brings them back. This is deliberately unlike the undo stack above, where a lower number really does discard. **Clear List Now** forgets every remembered project immediately: it does not wait for you to accept the dialog, and Cancel will not bring the list back. The same command sits at the bottom of the submenu itself; the button here exists so that clearing stays reachable when the submenu is switched off.

Auto-Save switches automatic saving on or off and sets how often it runs, every 5 minutes by default. It exists because most index editing lives in memory until you save (3.6), and a crash takes anything unsaved with it. An automatic save is the same operation as `Ctrl+S` in every respect, which has one consequence worth stating plainly: it clears the session backups too, so **Discard reverts to the last save, automatic or manual**, not to the moment you opened the project.

Three things keep it out of your way. The clock restarts whenever you save yourself, so the interval always means “this long since the last save”. A tick with nothing to write does nothing at all, and in particular does not move the Discard point just because the timer fired. And a tick that arrives at an awkward moment — a project still loading, an RTF export compiling, a dialog open, a table cell edit half-finished — is skipped rather than forced, leaving the next one to pick it up. Nothing about auto-save ever raises a dialog, on success or failure: a successful save is a brief status-bar note, and a failed one leaves your changes pending exactly as they were for the next attempt.

Session log folder names the folder the editor writes its session log into, created inside the open project's own directory. This is a folder name, not a path. The default is `session_logs` — deliberately visible, since the log is there to be read, or sent on, when something goes wrong.

Page number styles controls which encap names the Entry Table's Page column renders as bold or italic, so a cell shows the style its page number will actually print in. The defaults cover the standard LaTeX names (`bold`, `textbf`, `bf` and `textit`, `it`, `italic`). Add your own if your project styles page numbers with a custom command — a Table of Authorities using

`\authority`, say. Separate names with commas and leave off the backslash. Without this, a page number carrying a command the editor does not recognise looks like an ordinary editable cell, with no hint that it is styled at all. Whatever you leave in a field is the list, so removing a name stops it being recognised; clearing a field entirely restores that list's defaults rather than switching the styling off.

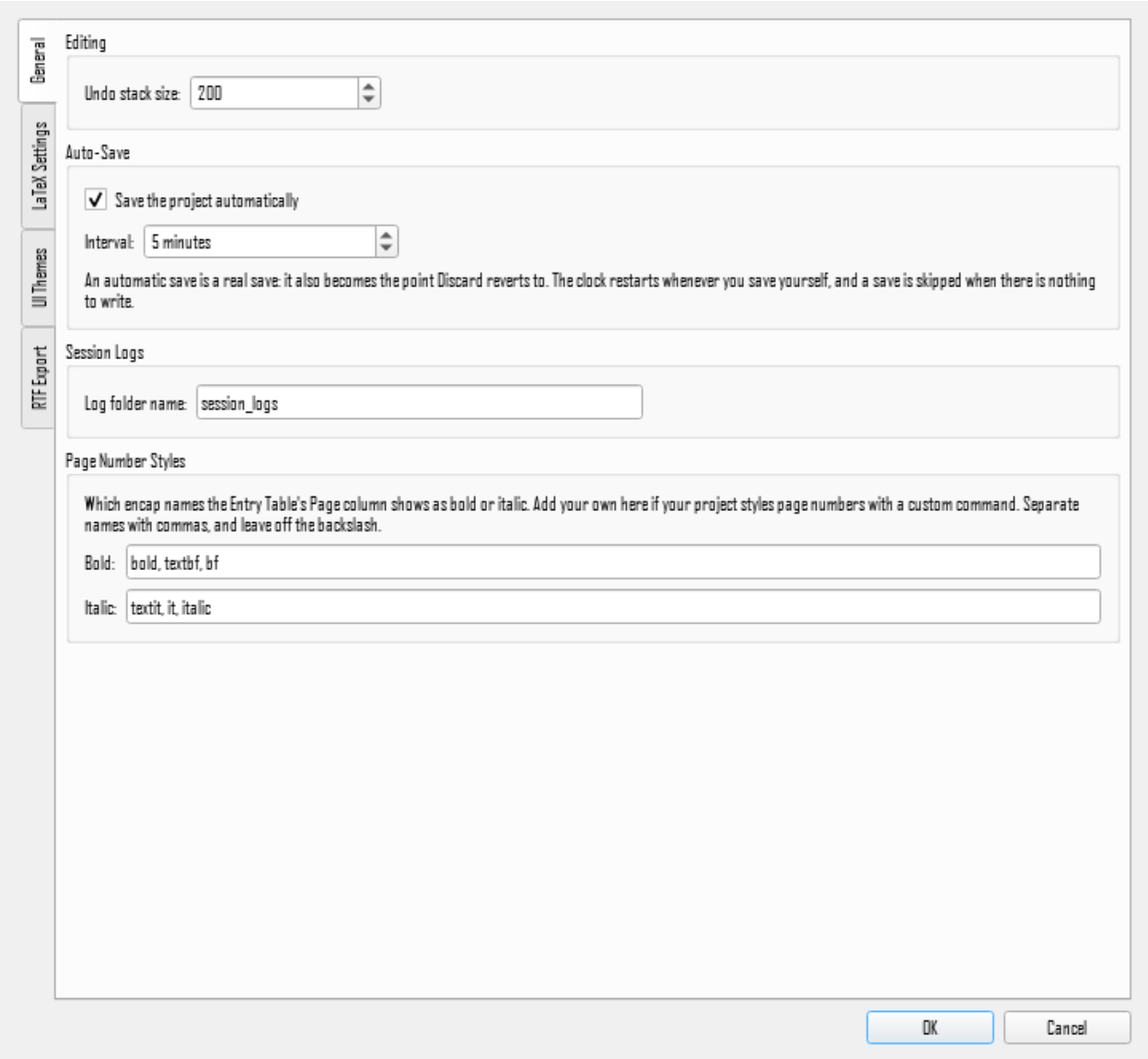

Figure 41. Preferences, General tab.

LaTeX Settings Overview

This tab is configuration only, changes here updates what the editor has stored; nothing is written to any `.tex` file. **Edit → Insert LaTeX Index Settings...** is the separate, explicit action that actually splices the corresponding text into your project's **base file**. This command inserts the generated `\usepackage/\makeindex[...]` lines immediately before `\begin{document}`, and the `printindex` call immediately before `\end{document}`. It's only available once a project is open with a **base file** chosen.

It's worth having these set the way you want before running that action for the first time, since the six sub-tabs below map directly onto what gets written. However, it's not a one-time, irreversible choice. Re-running **Insert LaTeX Index Settings...** later finds and replaces its own previously-inserted block rather than duplicating it, so revisiting your settings and re-inserting is always safe.

The six sub-tabs are, in order;

- **LaTeX Compiler**: locates the executable that typesets your document
- **imakeidx**: the package that actually builds the index
- **idxlayout**: column layout for the printed index
- **hyperref**: clickable page-number links
- **Index Engine**: choosing and configuring `makeindex` or `xindy`
- **Printindex**: the command that prints the compiled index

LaTeX Compiler

Enter the full path to your LaTeX compiler executable (Figure 42), either pdflatex, XeLaTeX (`xelatex`), or LuaLaTeX (`lualatex`). Browse opens a file picker with filters for each compiler, or type the path directly. `pdflatex` is the default, fastest, and most widely supported; choose XeLaTeX or LuaLaTeX instead for a project that needs system-installed fonts or native Unicode text. The compiler is required for **RTF Export**, left empty, that tool refuses to run rather than failing partway through.

If you're not sure which compiler a project actually expects, check the top of its base file for a line like

`%!TEX TS-program = xelatex`.

This is an IDE directive (recognized by TeXShop, TeXWorks, TeXstudio) naming the compiler to use; if present, match the compiler to this directive.

Unlike most settings in this chapter, *this path is machine-specific*, it points at a file on your computer, so it's stored separately from the rest of your preferences. It still follows the same global-vs-per-project behavior described in Global vs. Per-Project Settings below.

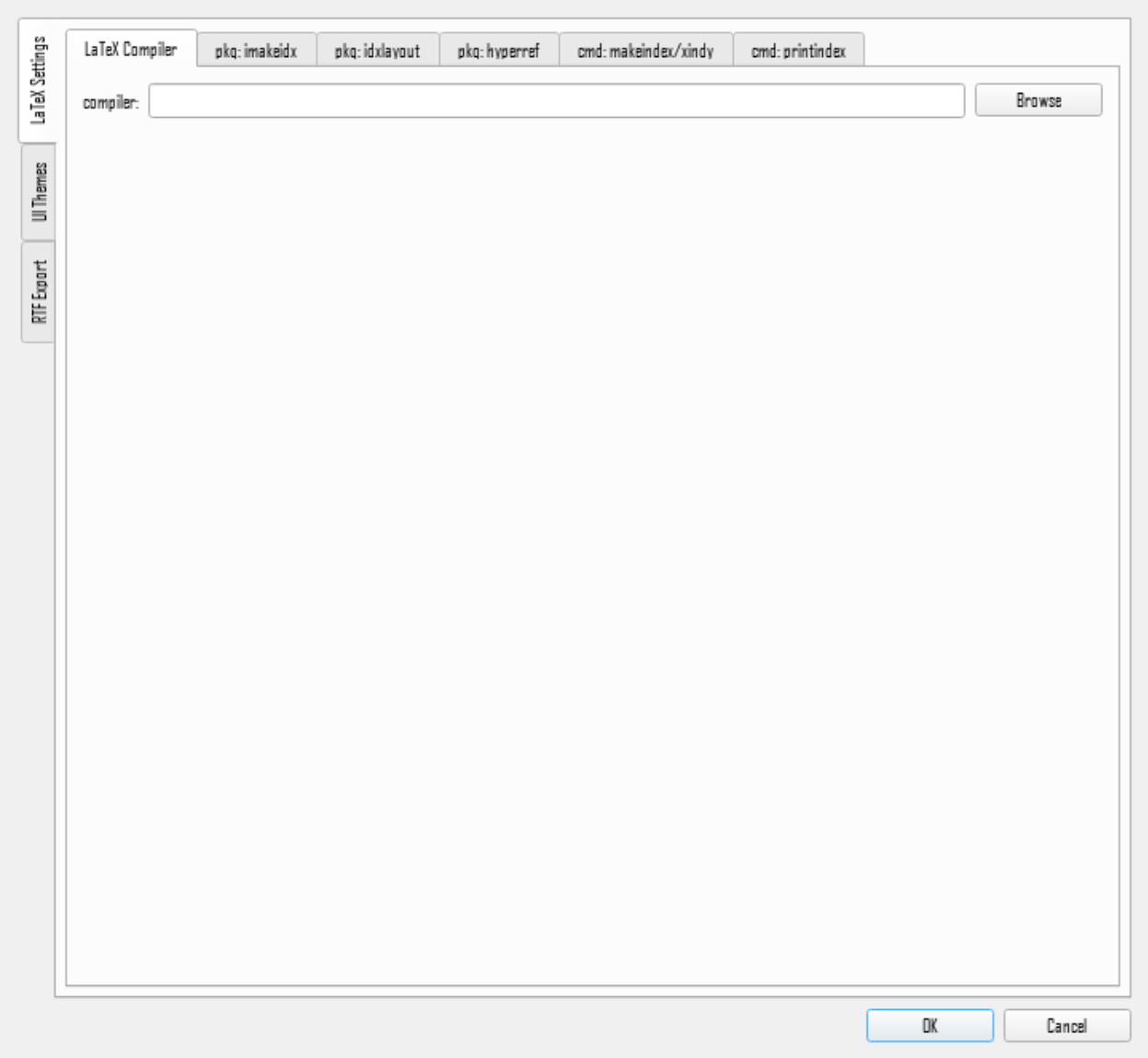

Figure 42. Preferences, LaTeX compiler tab.

imakeidx Package

`imakeidx` is the package that actually builds the index, every `\index` entry ultimately feeds into it, and this tab generates the `\usepackage{imakeidx}/\makeindex[...]` lines in your preamble (Figure 43).

- **Enable imakeidx package** is the master switch; everything else on the tab is grayed out and has no effect when it's off.
- **No Automatic Compilation** (`noautomatic`), on by default, stops `imakeidx` from trying to run the index engine itself during compilation, since **RTF Export** runs it on your behalf instead.
- **Prevent New Page Before Index** (`nonewpage`) keeps the index running directly on from whatever precedes it rather than forcing a page break
- **Number of Columns** (1-4, default 2) sets the printed index's column count

- **Index Title/Heading** overrides the index's printed heading (via `\makeindex[title=...]`). Leave it blank to use the document class's default `\indexname` heading, usually “Index.”
- **Add Index to Table of Contents** (`intoc`) adds that heading as a Table of Contents entry

If **Index Engine** is set to `xindy` instead of the default `makeindex`, an extra `xindy` option is added to the generated `\usepackage{imakeidx}` line automatically. There's nothing to configure for that here, it follows entirely from the engine choice on the **makeindex / xindy** tab.

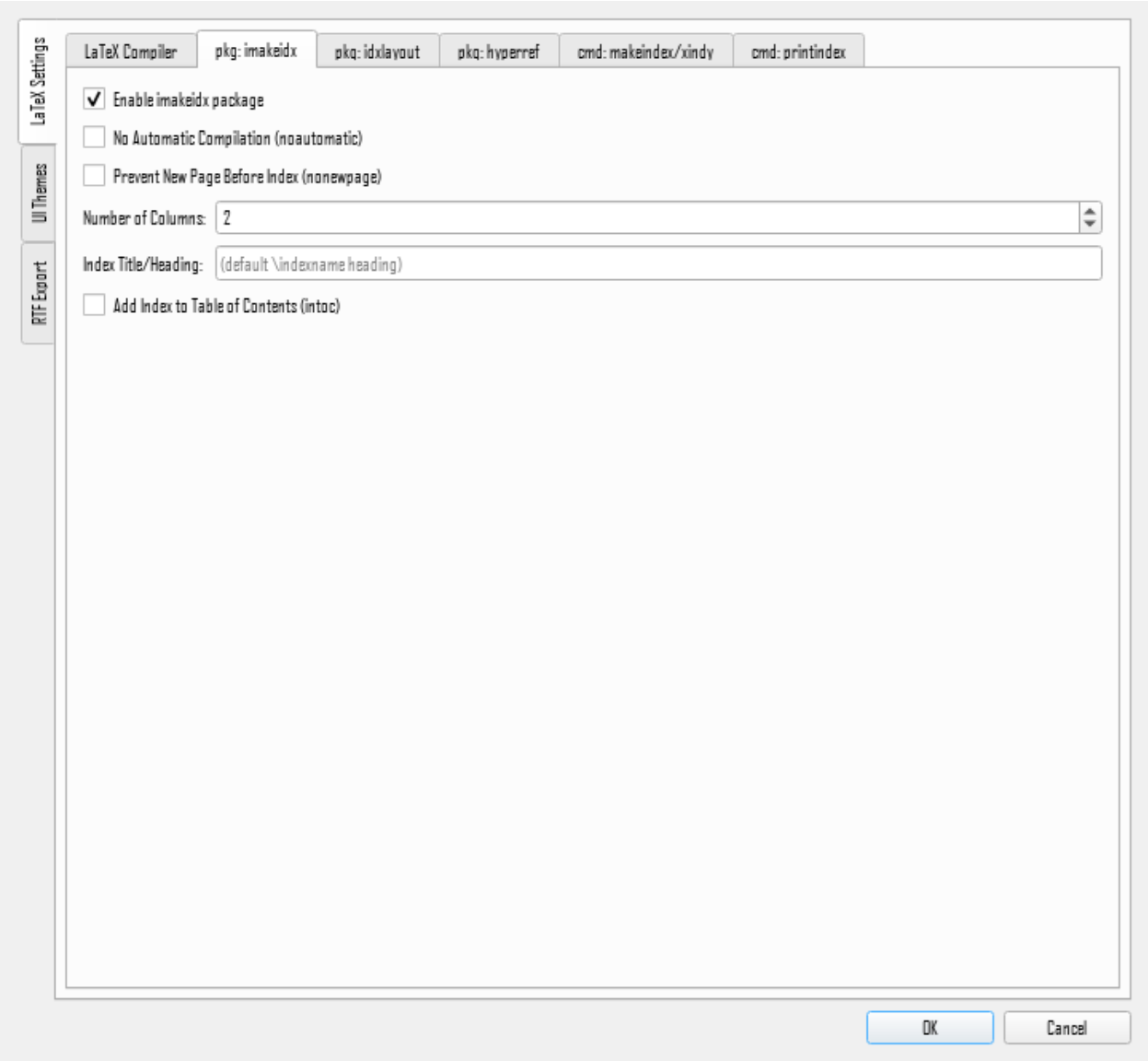

Figure 43. Preferences, imakeidx tab.

idxlayout Package

`idxlayout` is an optional package giving finer control over the printed index's multi-column layout (Figure 44). It is for use alongside `imakeidx`, not instead of it.

- **Enable idxlayout package** is the master switch.

- **Allow Unbalanced Columns** (`unbalanced=true`, on by default) lets the last column end up shorter than the others instead of the package stretching earlier ones to balance them out.
- **Justified Columns** (`justified=true`, off by default) justifies each column's text instead of leaving a ragged right edge.

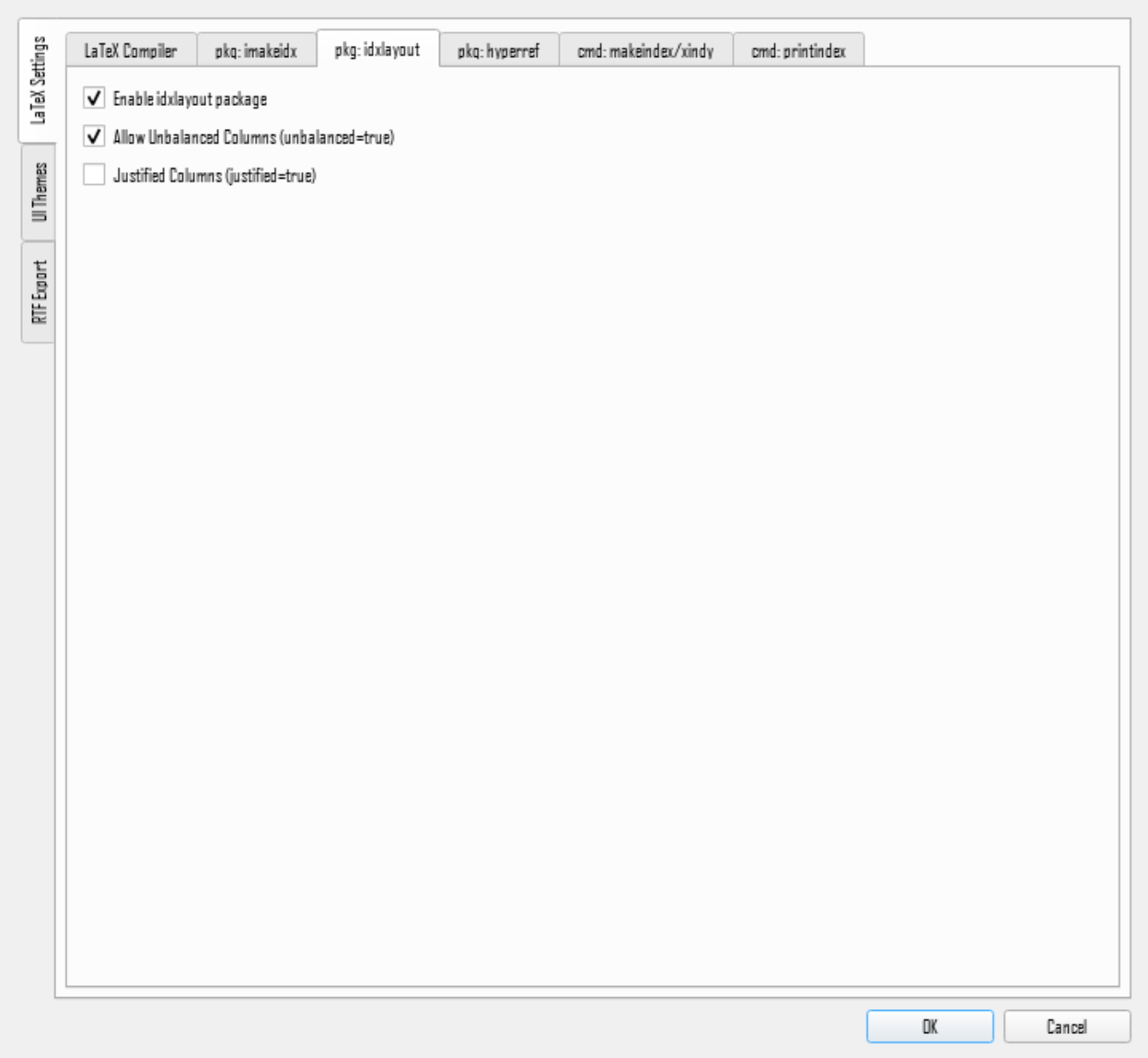

Figure 44. Preferences, idxlayout tab.

hyperref Package

`hyperref` makes an index entry's page number a clickable link back to that page in the generated PDF (Figure 45).

- **Include hyperref linkage** is the master switch — off by default. Checking it adds a `\usepackage{hyperref}` line to your preamble (with whatever options below are enabled); everything else on the tab is grayed out and has no effect while it's off

- **Colorized Links** (`colorlinks`, on by default) makes linked page numbers appear in color instead of a boxed border. The usual way hyperref links look in a printed or PDF document
- **Link Target Color** sets that color: blue (default), red, black, or magenta.

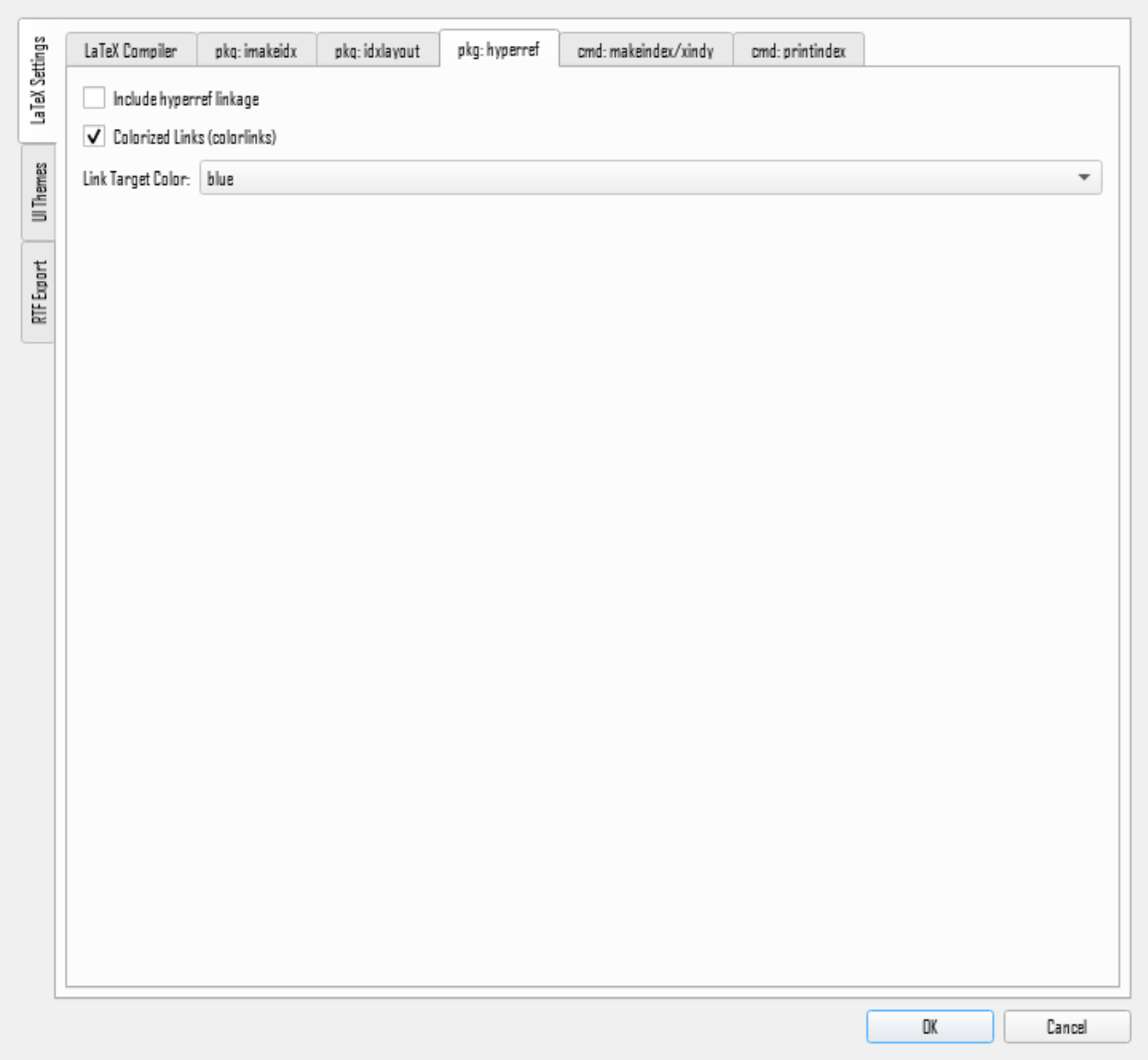

Figure 45. Preferences, hyperref tab.

Index Engine (makeindex / xindy)

This tab configures the program that actually turns your raw `\index` entries into a sorted, formatted index (Figure 46). `makeindex` amd `xindy` are both standard parts of a normal TeX Live or MiKTeX install.

Core configuration

- **Execution Command Binary** chooses the engine: `makeindex` (the traditional, simpler tool, the default) or `xindy` (more flexible, better multi-language/sorting support). Switching it shows or hides the engine-specific options below to match.
- **Executable Path** is that engine's own executable. Browse to pick it;[20] switching engines clears this field automatically, since a path chosen for one engine isn't valid for the other. Both the engine choice and this path are required for **RTF Export**.

makeindex options (shown only when `makeindex` is selected),

- **Compress Intermediate Blanks** (`-c`, on by default) collapses multiple consecutive spaces within an index key into one
- **Ignore Leading Spaces** (`-p`, off by default) ignores spaces at the very start of a key when sorting
- **Sort Ordering Rule** chooses word-by-word or character-by-character sorting
- **Target Stylesheet Name** is the filename for `makeindex's` generated `.ist` style file

xindy options (shown only when xindy is selected),

- **Language Module** (`-L`) picks sorting/collation rules for a language, English, French, German, Ngerman, Spanish, or Italian
- **Input Encoding** (`-C`) is the expected character encoding
- **Markup Language** (`-I`) tells `xindy` the index uses LaTeX markup, normally left at latex.
- **Allow Duplicate Page References** lets the same page number appear more than once for an entry rather than collapsing repeats.
- **Target Module Name** is the filename for `xindy's` generated `.xdy` module file

Index Formatting Rules are shared by both engines, the same choices get translated into whichever engine's own syntax (`.ist` for `makeindex`, `.xdy` for `xindy`), so switching engines doesn't require reconfiguring them:

- **Enable Alphabetical Section Headers** groups the index into A/B/C… sections
- **Render Letter Headers Bold** bolds those headings
- **Use Dot Leaders** connects an entry to its page number with a dotted line
- **Non-Alphabetic Symbols Labe**l the label name for section containing symbol-first entries
- **Numeric Entries Label** the label name for section containing digit-first entries
- **Standard Page Delimiter Mapping** is the text between an entry and its page number
- **Page Range Connection Symbol** joins the start and end of a page range. Defaults to double dashes (--), which is translated into an en-dash (--)

[20] Like the path for the LaTeX engine, this path is machine-specific.

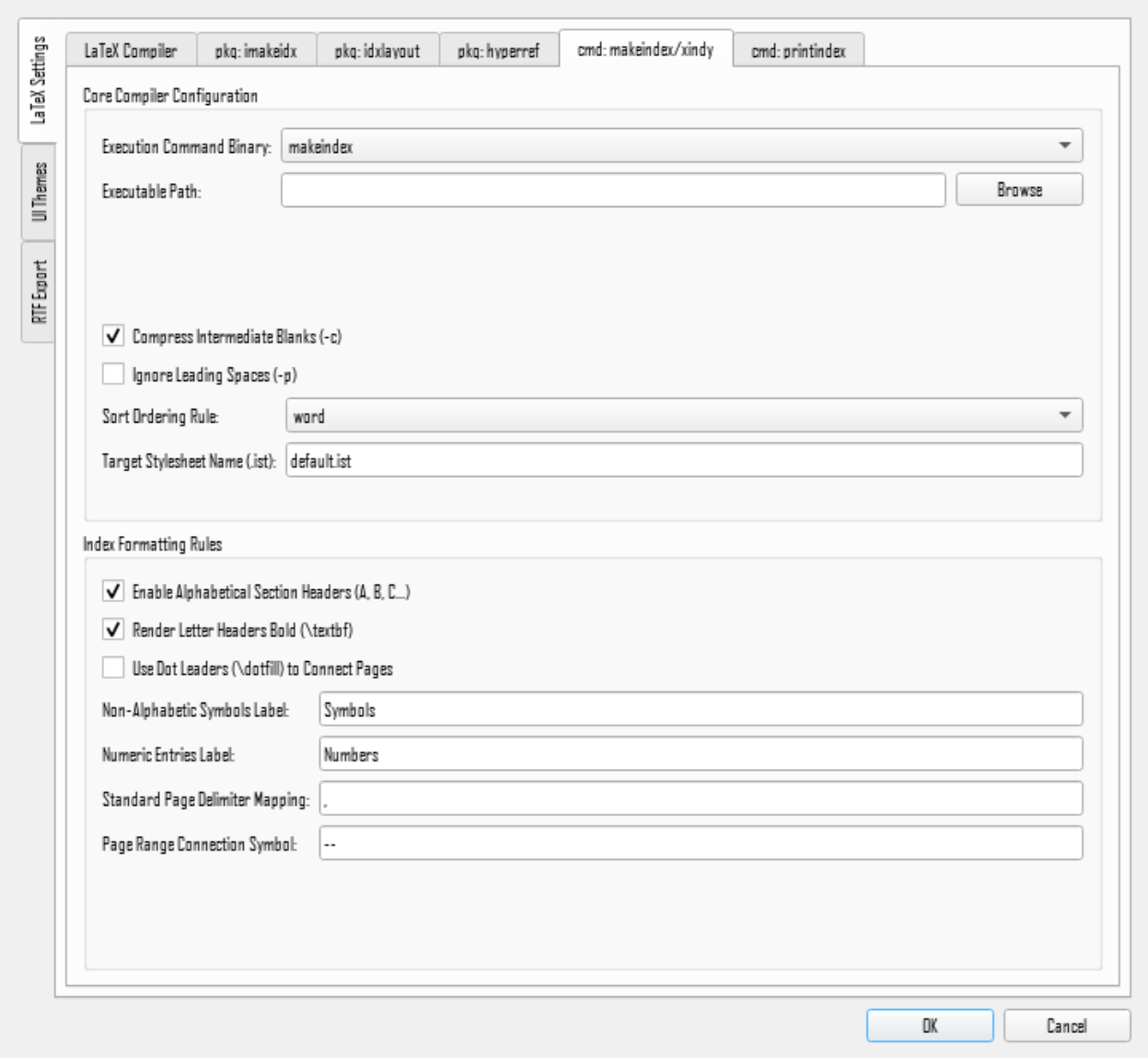

Figure 46. Preferences, makeindex / xindy tab.

printindex Command

The command that actually prints the compiled index into your document, and how it's laid out (Figure 47).

Output Printing Command is the bare command name (no backslash) that prints the index, normally left at the default `printindex`, matching `imakeidx's` own `\printindex`. Only change this if your document uses a differently-named custom command.

Wrap inside Multicols environment block wraps the `printindex` call in a multicols environment, using the same column count set on the `imakeidx` tab (10.1.2), instead of relying on `imakeidx's` own column handling.

The command name matters beyond this tab, it's also what **Head Notes** looks for when deciding where to insert `\indexprologue{...}`, since a head note has to land immediately

before whatever actually prints the index. Renaming this command here means **Head Notes** still finds and anchors to it correctly, without any extra step.

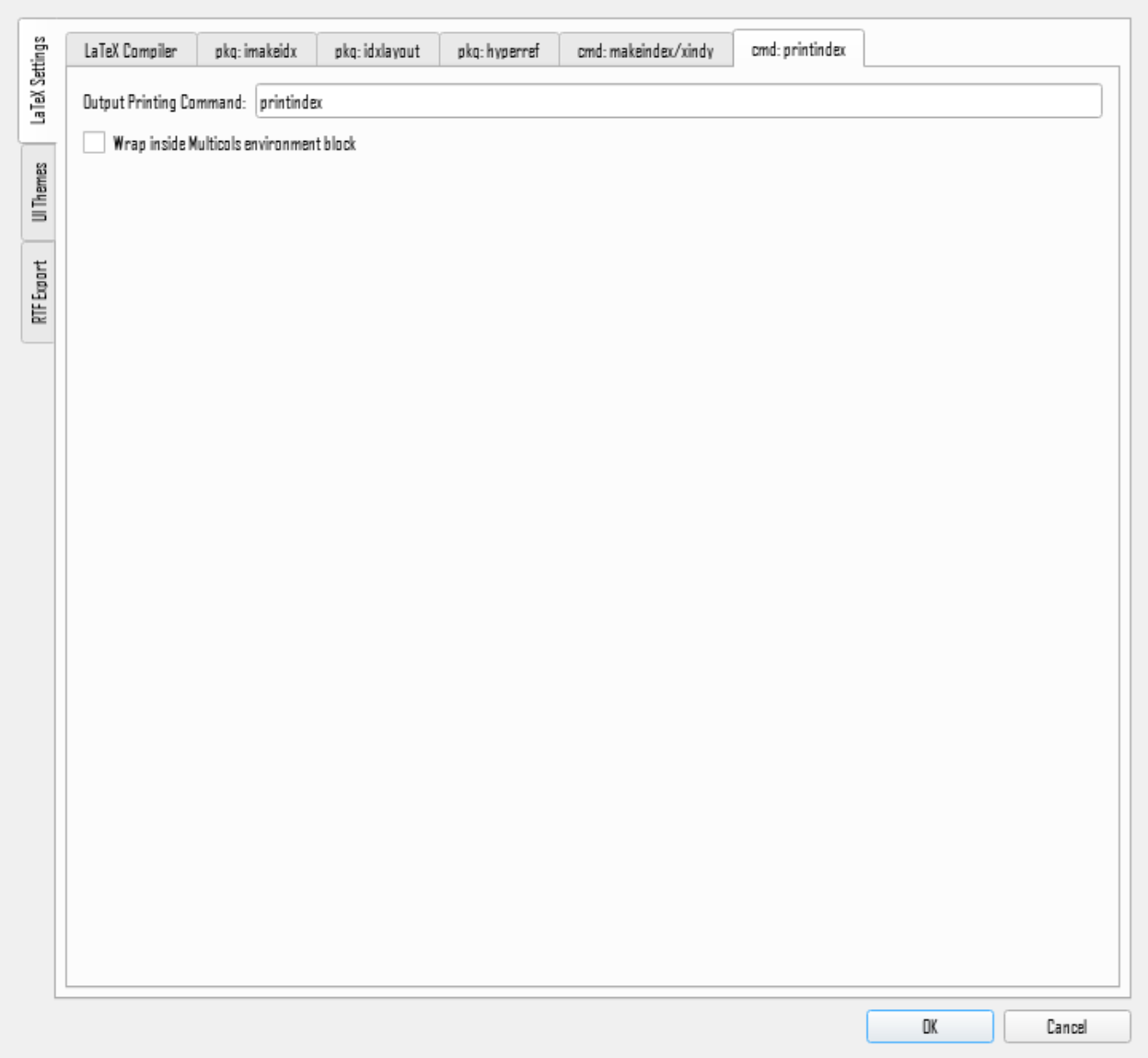

Figure 47. Preferences, printindex tab.

UI Themes

Separate colour editors for **Dark Theme** and **Light Theme** including; window background, text, highlight colour, tab pane background, and so on (Figure 48). Each is a click-to-pick colour swatch with a live preview alongside it. This tab is about the colour palette each theme uses, not which one is currently active: switching between dark and light mode itself is accomplished with the toolbar toggle

Restore Tab Defaults on each tab resets that theme back to its factory colours. Colour changes apply immediately across the whole application as soon as you accept the dialog, no restart needed.

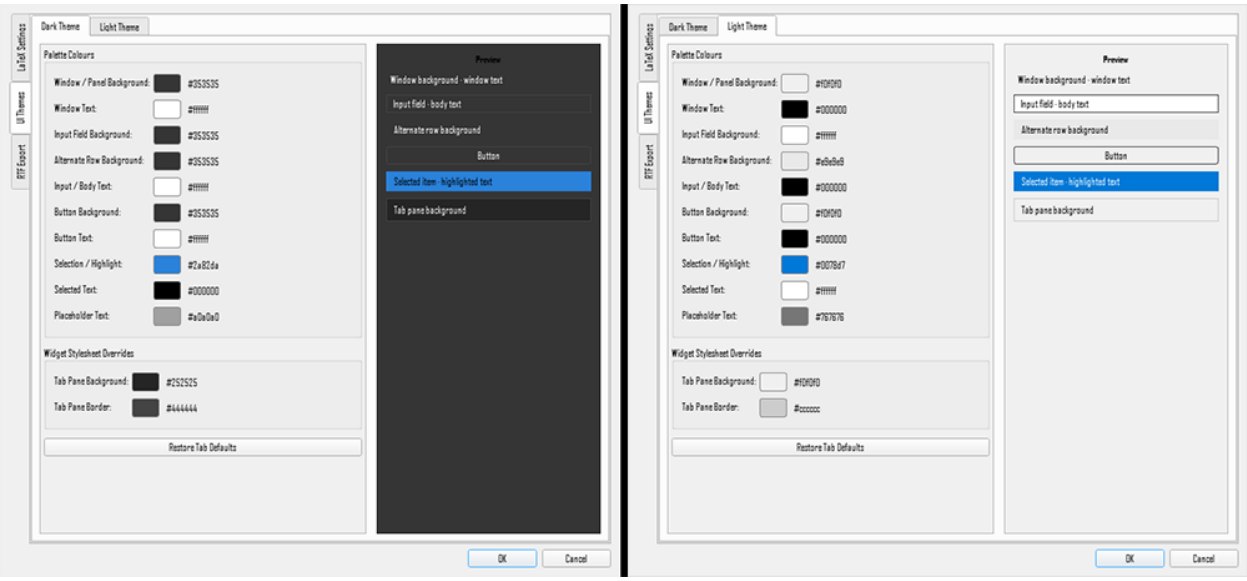

Figure 48. Preferences, UI themes tabs.

Global vs. Per-Project Settings

Changing Preferences without a project open modifies the global default settings for the application. The toolbar controls for Font family/size and the dark/light mode toggle are globally shared settings with no project-specific variants. Everything on the General tab is global in the same way, with no per-project copy.

The first time a project is opened, it inherits the global defaults; from then on, that project's own copy is completely independent, changing Preferences with a project open only affects that project, never your global defaults or any other project's settings.

The LaTeX compiler and index engine executable paths are machine-specific rather than stylistic, so they're stored in their own dedicated place rather than alongside the rest of your preferences. But they still follow the same per-project rule as everything else, open a project for the first time and it inherits your current global path, then keeps its own independent copy from then on.

Declared Alphabets

Most indexes file by the ordinary Latin alphabet and the application does that without being asked. Some languages do not. Welsh treats ch, dd, ff and ng as single letters filing after c, d, f and g; the Mayan alphabets place glottalised letters after their plain forms. Filing such an index by the English alphabet is not a matter of taste, and it is wrong in a way that looks perfectly tidy on the page.

Preferences → Sorting is where you choose a declared alphabet and where you write your own. Three are supplied, Welsh, Yucatec Maya and Guatemalan Maya, each transcribed from a named authority which the page shows beside it.

The choice is per language rather than per project. An index mixing Welsh place names with English subject headings applies the Welsh order to the Welsh entries only, which is not a refinement: a project-wide declaration was tried and it reordered the English headings standing beside the Welsh, because a rule that fires on th fires on another and the as readily as on Aberddawan.

Writing your own asks for the letters and nothing else. Type them in the order your authority prints them and the application works out the filing rules. As you type it tells you two things: letters that file out of the order you typed, and letters that file correctly but will print under another letter's heading. Neither stops you saving. Both exist so you find out while you can still do something about it.

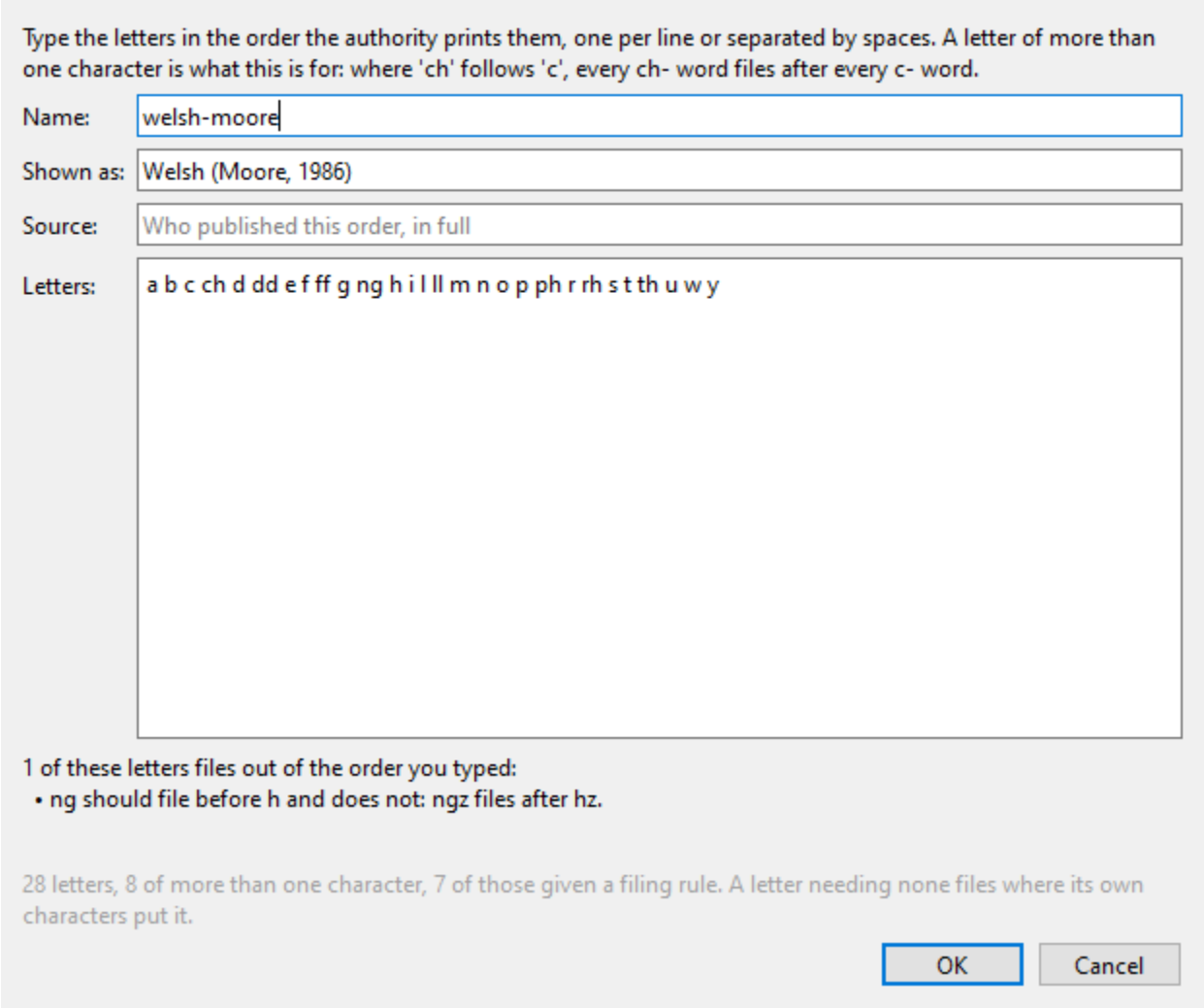

Figure 49. Writing a declared alphabet. The editor names the letters this mechanism cannot place as they are typed.

What Kind of Index Is This?

Preferences → Sorting opens with a question rather than a setting: what kind of index this project is. Subject, name, title, authorities or first line. The answer seeds every filing rule below it, because a name index and a subject index do not file alike and neither of them files like a table of authorities.

Declaring is an event and not a mode. Choosing a kind fills the settings below with that kind's defaults once, and from then on they are this project's to edit. Nothing re-applies them behind you, which is worth knowing when you read a setting back: an option that is off means you switched it off, not that it was never seeded.

And a declaration never takes a setting away. Declaring a name index and then declaring a subject index leaves the name index's prefix lists in place, because seeding adds and does not remove. If you want those lists gone you clear them yourself, which is a good deal better than the application silently emptying six lists you may have spent an afternoon editing.

A project that has never been asked is a subject index, which is the commonest answer and the one that changes least. If your index is one of the other four, declaring it is the single most useful thing you can do to its filing, and it takes one choice.

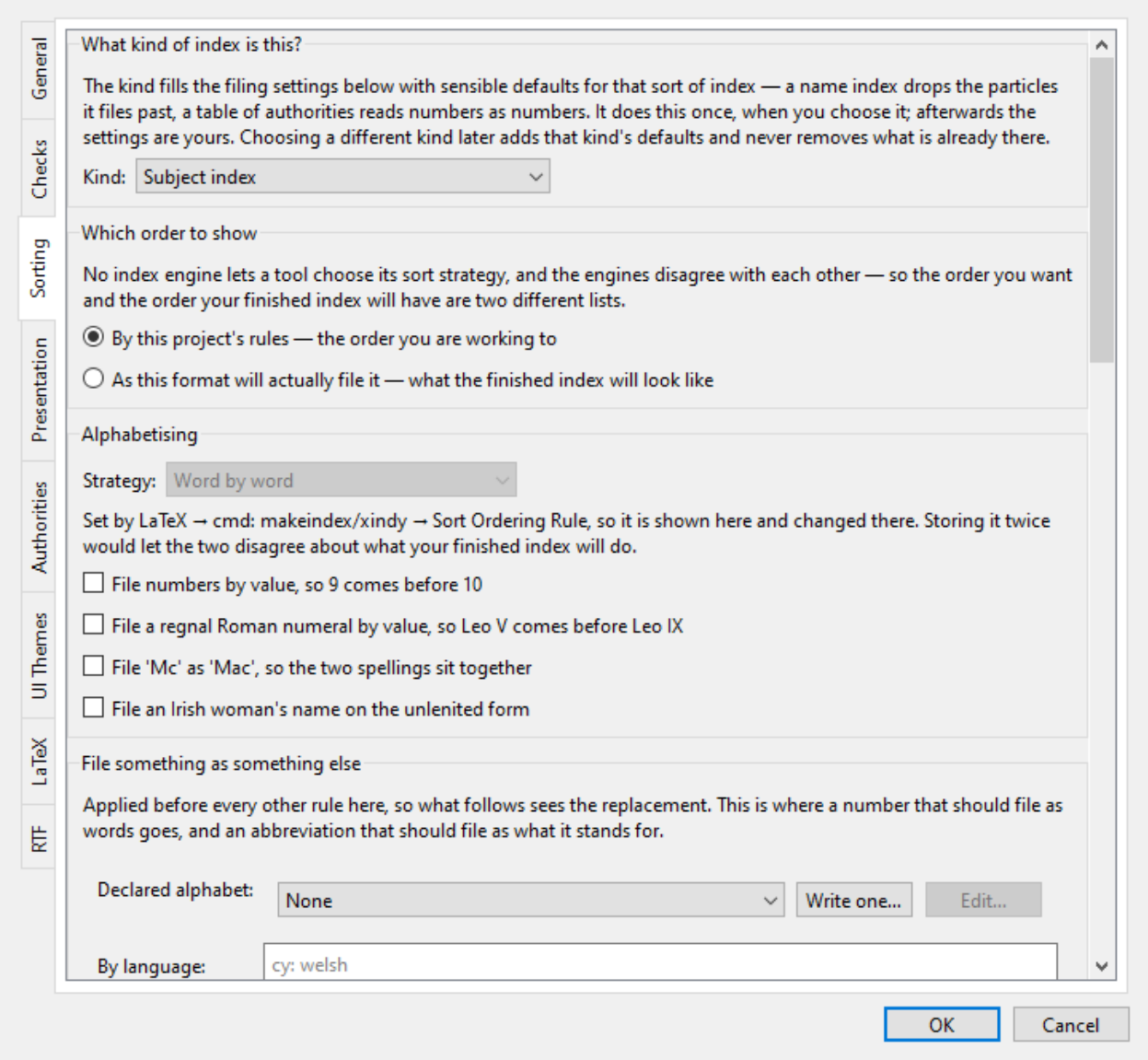

Figure 50. Sorting preferences. The kind declared at the top seeds everything below it.

Chapter 12

Troubleshooting

This chapter collects common problems encountered by indexers working with LaTeX.

Extra Space Around Styled Text in Captions, Footnotes, or Headings

Symptom: An entry whose display text includes a formatting command (`\textit{...}`, `\textbf{...}`, and similar) picks up a stray extra space in the printed index, but only when the `\index{...}` macro that produced it sits inside a caption, footnote, section heading, or other "moving argument." The result is two separate-looking entries in the index where there should be one, for example "Miscellanea Analytica" and "Miscellanea Analytica " (with an invisible trailing space)[21] sorted as if they were different terms. Plain-text entries, with no formatting command in their display text, are never affected. No error or warning appears anywhere, the only sign of the misplaced space is in the compiled index itself, which is why this one is easy to miss until the index is already typeset.

Cause: This is a genuine LaTeX quirk, not an editor or `makeindex` bug: LaTeX silently inserts a space after a formatting command when it copies a "fragile" argument out of a moving argument. It's a long-standing bug and is undocumented in the usual references. It was reported to the LaTeX bug tracker in 1995 and never fixed at the LaTeX level, since a proper fix risks breaking more than it repairs.

Fix, in order of preference:

1. **Best — move it out entirely.** Create the entry from a location outside the moving argument if at all possible; this sidesteps the problem completely rather than working around it
2. **For a figure or table specifically,** keep the index insertion point inside the figure/table environment, so it still floats with and gets the correct page number, but position it outside the `\caption{...}` block
3. **If the entry genuinely must live inside the moving argument,** prefix the formatting command with `\string` in the entry's display-text field.
   `De Revolutionibus@\string\textit{De Revolutionibus}`
   Type this directly into the Display field in the **Entry Table** or the Main/Subhead field in the **Index Entry** window, since those fields accept raw LaTeX in addition to plain text

"My Entry Disappeared" — the Three-Level Heading Limit

Symptom: An entry compiles cleanly with no error or warning from LaTeX or the index engine, but is simply missing from the printed index.

Cause: A raw `\index` tag with more than two `!` separators (more than three heading levels) is silently dropped by the indexing engine. This is a hard limit of the underlying tool, not a bug:

```
\index{main!sub1!sub2!sub3}   % four levels -- silently dropped, nothing printed
```

This is exactly why the editor's **Entry Table** and **Index Entry** window expose precisely three heading fields; **Main**, **Subhead 1**, **Subhead 2**, there is no valid LaTeX construct beyond that depth to expose.

Caution: because the application builds the final heading by joining those three fields with `!` internally, typing a literal `!` character into the text of one of the three fields does not create a usable fourth level, it just introduces an extra, unintended split point that the indexing engine parses the same way, silently dropping the entry. Typing "`Smith! Jones`" into the Main field, for instance, is read by `makeindex` as two heading levels ("`Smith`" and "` Jones`"), not as a single

[21] The space is eliminated in the formatted index generated in the PDF by `\printindex`, resulting in two separate entries in the index that look identical. The trailing space is only visible in the intermediate `.ind` file generated during index creation.

heading containing an exclamation point. A term that legitimately contains an exclamation point, or a hierarchy that genuinely needs a fourth level, can't be represented directly, the latter requires restructuring the heading hierarchy rather than a workaround.

To index a heading that genuinely contains an exclamation point (or other special character), escape it using the conventions in Escaping Special Characters.

Catching Inconsistent Entries (Singular/Plural, Spelling, Punctuation)

A large, sloppy-looking index is usually not wrong markup but inconsistent wording: “gnat” vs. “gnats,” “Hewlett-Packard” vs. “Hewlett Packard,” “Adobe Systems, Inc.” vs. “Adobe Systems Incorporated”, each variant sorts and displays as its own separate entry instead of merging into one. LaTeX and the indexing engine have no way to catch this automatically since, as far as they're concerned, these are validly different strings.

A less obvious variant of the same problem: stray spacing. Unlike ordinary LaTeX text, `makeindex` does not compress multiple spaces or ignore a leading/trailing one inside an `\index` argument, so entries that look completely identical when printed can silently sort as separate, “duplicate” items:

```
\index{multiple words}
\index{ multiple words}
\index{multiple  words}
\index{multiple words }
```

All four look identical when printed, but `makeindex` treats them as four distinct sort strings, so the printed index can end up with the same-looking heading listed two or three times with different page numbers under each. This can just as easily happen through the editor if a Main/Subhead field is typed or pasted with a stray leading or trailing space — it's worth getting in the habit of checking for one if a heading you expect to be merged shows up twice in the **Index Tree** instead.

The application does not currently automate detection of either kind of inconsistency, **Range Consistency Check** validates range structure specifically, and **Index Statistics** gives entry counts, but neither compares wording or spacing across entries for near-duplicates. The **Index Tree** helps spot these errors, since it keeps every heading sorted alphabetically at all times, so spelling, pluralization, punctuation, and spacing variants of the same term land visually near each other and are much easier to spot by eye than they would be scanning raw `\index{...}` macros in the `.tex` source. Before a final compile, it's worth scrolling the **Index Tree** end to end once, specifically looking for near-duplicate neighboring nodes.

As a matter of working habit: settle on one spelling/punctuation form for a term the first time it's indexed, and reuse it consistently rather than retyping it from memory each time (the auto-completion feature in the **Index Entry** window helps with this). Duplicating an existing entry via **Duplicate References** to cross-post it under a different heading is exact-copy-safe in a way retyping from memory is not.

“Missing $ inserted” — A Malformed Page Range

Symptom: Somewhere in the final `latex` run, the one that reads the compiled `.ind` file to typeset the actual index, compilation stops with “`Missing $ inserted.`”, and the error points at a line inside the `.ind` file rather than anywhere in the `.tex` source, which makes it disorienting to track down.

Cause: Two `|(` range openers for the exact same heading text, with no intervening `|)` closer between them:

```
\index{gnat|(}
...
\index{gnat|(}   % <- a second opener for "gnat" before the first one closed
...
\index{gnat|)}
```

`makeindex` has a long-standing bug in this situation: instead of rejecting the second, unmatched opener outright, it converts it to a literal `\(` (LaTeX's inline-math-open command) followed by the page number in braces, and writes that straight into the `.ind` file. The next LaTeX run reads `\(` as the start of an in-line math expression it can never find the end of, which is what actually produces the "`Missing $ inserted.`" error.

Before it ever reaches that point, `makeindex` itself will already have reported the underlying problem, running `makeindex` produces a warning in the `.ilg` log file:

```
## Warning (input = gnat.idx, line = ...; output = gnat.ind, line = ...):
-- Extra range opening operator (.
```

Treat that `.ilg` warning, not the eventual "`Missing $ inserted.`" error two steps later, as the real point to catch the problem, by the time the error appears the broken `.ind` file has to be deleted and regenerated before LaTeX can even finish the run that reports it.

Fix: this exact shape of problem is what **Range Consistency Check** is built to find. Its "Malformed ranges" category catches an opener with no matching closer; its "Overlapping ranges" category catches a second opener for the same heading starting before the first one's range has closed, which is precisely the situation above. Run it and apply the suggested fixes before compiling, rather than tracking a "`Missing $ inserted.`" error back through the `.ind` file by hand. The check operates directly on your project's entries, not on generated output, so once it reports clean this class of error can't occur at compile time.

Page Numbers Are Off By One

Symptom: An index entry cites a page number that's one, occasionally two, higher or lower than the page where a reader would actually expect to find the reference.

Cause: Because a `\index` macro is invisible in the typeset output, its page number is simply whatever page LaTeX happens to be laying out at that exact point in the source, so if that point falls right at a page boundary, which side it lands on can be one page off from where a human reader would place it. A few situations make this especially likely:

New chapters and sections: an entry referring to a new chapter or section, entered before the `\chapter{...}/\section{...}` line itself, will typically get a page number too low by one (occasionally two, depending on where the previous section ended). Enter it just after the heading instead. Right after a `\label`, if the document uses one, is a reliable spot.

Theorem-like environments: the same applies to a definition, theorem, or similar numbered environment, an entry placed before the environment begins can be off by one if the environment starts a new page. One placed after the environment ends can be off by one in the other direction if the environment ends a page.

Displayed equations: an entry placed after a display can be too high by one if the display itself ends a page. Place the entry before the display instead, with no blank line in between, so at least one line of ordinary text precedes the display on whichever page it starts.

Multi-word terms and personal names spanning a page break: if the words a term is attached to are themselves split across a page break, which side the entry lands on depends on which specific word you attached it to. Attach it to the word a reader is actually most likely to look for — for a name like “Carl Friedrich Gauss,” that's the surname, so:

```
Carl Friedrich Gauss\index{Gauss, Carl Friedrich}
% entry attached after "Gauss" -- safer

Carl Friedrich\index{Gauss, Carl Friedrich} Gauss
% attached after "Carl" -- if a page break falls
% right here, "Gauss" won't be on the cited page
```

None of this is something the application can detect or correct automatically, a page number is only ever “wrong” relative to a human reader's expectation of where they'd look, not to any rule LaTeX or `makeindex` can check. The only real defense is where you place your cursor (or your text selection, for a range) before inserting an entry as close as possible to the specific word or line a reader would scan for, and on the correct side of any nearby page, section, or environment boundary.

Appendix

This appendix is quick-reference material. A is a syntax cheat sheet mirroring Chapter 2 one-for-one; B is the complete keyboard shortcut tables, C is the database table summary, and D is a glossary of terms used throughout this guide.

A. \index Syntax Quick Reference

Every raw `\index` form introduced in Chapter 2, with a bare example of each; `term`, `main`, `sub1`, `sub2`, `target`, `sortkey`, and `displaytext` stand in for whatever text you'd actually write.

Form	Example
Basic entry	`\index{term}`
Sub-entry (up to 3 levels)	`\index{main!sub1!sub2}`
Page range — opener	`\index{term|(}`
Page range — closer	`\index{term|)}`
“See” cross-reference	`\index{term|see{target}}`
“See also” cross-reference	`\index{term|seealso{target}}`
Sort key vs. displayed text	`\index{sortkey@displaytext}`
Bold page number	`\index{term|textbf}`
Italic page number	`\index{term|textit}`
Escape a literal `@`	`\index{term"@more}`
Escape a literal `!`	`\index{term"!more}`
Escape a literal `|`	`\index{term"|more}`
Escape a literal `"`	`\index{term""more}`

Remember that ranges, cross-references, sort keys, and page-number styling all have their own controls in the editor, the forms above are what ends up in the `.tex` file, not something you're expected to type by hand, aside from the escaping rows, which the editor does not currently automate.

B. Keyboard Shortcuts Tables

Every shortcut in the application, grouped by where it applies. Menu shortcuts also appear next to their command in the menu itself.

File menu

Shortcut	Action
Ctrl+O	Open Project...
Ctrl+S	Save Project
Ctrl+W	Close Project
Alt+F4	Exit

Edit menu

Shortcut	Action
Ctrl+F	Find...
Ctrl+Shift+F	Advanced Search...
Ctrl+,	Preferences...

View menu

Shortcut	Action
Ctrl+B	Focus File Pane
Ctrl+Shift+I	Focus Index Pane
Ctrl+E	Focus Edit Entries Pane
Ctrl+\	Toggle Index Entry Window

Tools menu

Shortcut	Action
Ctrl+Shift+H	Create Head Note...
Ctrl+Alt+C	Create LaTeX Command...
Ctrl+Alt+M	Manage Project Commands...
Ctrl+Alt+R	Create Rtf File

Resync Index Data from Disk, **Resync Workspace Files from Disk**, **Manage Pruned Files...**, **Index Statistics...**, and **Check Range Consistency...** don't have shortcuts, use the Tools menu.

Help menu

Shortcut	Action
`F1`	Contents...
–	About LaTeX Indexing Editor...

Global (works anywhere in the window)

Shortcut	Action
`Ctrl+Shift+D`	Toggle dark/light mode

Index Entry window

Shortcut	Action
`Ctrl+K`	Insert the index entry (same as clicking Insert Index Tag)
`Esc`	Close the window
`Tab` (autocomplete suggestions showing)	Accept the highlighted suggestion
`Backspace` in an empty sub-entry field	Clear the field and jump back to the previous one

Find dialog

Shortcut	Action
`Enter`	Find Next
`Shift+Enter`	Find Previous
`Esc`	Close the dialog

Editor tabs:

The read-only `.tex` file tabs restrict editing to a safe subset of keys: arrow/paging navigation, `Ctrl+C` (copy), `Ctrl+A` (select all), `Ctrl+F` (Find, same as the Edit menu), `Ctrl+Z` (undo), `Ctrl+Y` (redo), and `Esc` to clear the current selection. Everything else; typing, cut, paste, is blocked since these tabs aren't meant for freehand editing outside the index-entry workflow.

C. Database Table Summary

Every project has its own SQLite database, `<ProjectName>_index_manifest.db`, created alongside your .tex files the first time the project is opened.

Project database

Table Name	Description
`project_metadata`	Key/value store for project-level settings: schema version, project name, root .tex file, compiler and index-maker executable paths, and output directory.
`project_files`	Registry of every file discovered in the project, and whether it's currently active or pruned from indexing.
`project_headings`	One row per distinct index heading path. Separating headings from the occurrences that use them is what gives a heading a stable identity: a rename changes one row and every reference filed under it follows.
`project_references`	Every \index entry found in the LaTeX source: its location in the file, encapsulation, see/see also text, and range-pairing info for start/end page ranges.
`project_file_sync_state`	A checksum per file, recorded whenever the database is known to match what's on disk; used to detect edits made outside the editor between sessions.
`project_custom_commands`	Custom LaTeX indexing commands added to this project from the global command registry.
`project_cross_references`	The authoritative source for “see” and “see also” cross-references; cross_refs.tex is regenerated from this table whenever a cross-reference is added, edited, or removed.

One additional database is shared, and since September 2026 it is shared more widely than this application: it is kept once for the machine and reached by every application built on the same indexing core, so a name decision made here is available in the Word index editor too.

Name-authority cache

The shared store (DH Indexing\shared\indexing.db), in your local application data. It holds five tables; the one this application reads is below.

Table Name	Description
`cache`	Cached name decisions, so the same individual does not have to be inverted twice. The other four tables hold authored house styles, declared alphabets, language-model assessments, and the store's own schema version.

D. Glossary of LaTeX Indexing Terms

Term	Description
Base file / root file	the one `.tex` file in a project that actually gets compiled, containing `\documentclass{...}` and `\begin{document}`; every other file is pulled into it
Collation	the sorting process that turns raw, unsorted index entries into their final alphabetical order; performed by `makeindex` or `xindy`, never by LaTeX itself
Cross-reference	an index entry that points to another heading instead of listing a page number: “see” redirects entirely, “see also” supplements
Encapsulator	`makeindex's` term for the `|command` suffix attached to a heading, which wraps its page number in a formatting command or replaces it with a cross-reference pointer when the index is typeset.
Escape character	the quotation mark (`"`), which forces one of `makeindex's` five reserved characters to be read literally instead of triggering its special meaning
Fragile command	a LaTeX command that doesn't expand safely when used inside a moving argument without `\protect`; relevant to why formatting inside an index entry in a caption or footnote can misbehave
.idx file	the raw, unsorted list of index entries LaTeX writes out during compilation, one per `\index` macro
.ind file	the sorted, formatted index produced by running `makeindex` or `xindy` against the `.idx` file; what `\printindex` actually reads and typesets
Main entry / sub-entry / sub-sub-entry	the three heading levels a single `\index` argument can express, separated by `!`; the Main / Subhead 1 / Subhead 2 fields in the editor
makeindex	the most common program used to sort and format an `.idx` file into an `.ind` file; assumed throughout this guide unless `xindy` is mentioned specifically
Moving argument	a LaTeX construct (a caption, footnote, section heading, or similar) whose contents may be typeset more than once and are therefore restricted
Page range	a single index entry covering a span of consecutive pages rather than one page, written as a linked opener/closer pair of `\index` macros

Term	Description
Project database	the small SQLite file (`<ProjectName>_index_manifest.db`) the editor creates alongside your `.tex` files to track every entry it finds, without which reopening a large project would require re-scanning it from scratch every time
Sort key	an alternate string, given before an `@`, used to alphabetize an entry differently from how it's displayed
xindy	an alternative to `makeindex`, with more flexible support for non-Latin alphabets and custom sort rules; mentioned but not required by this guide